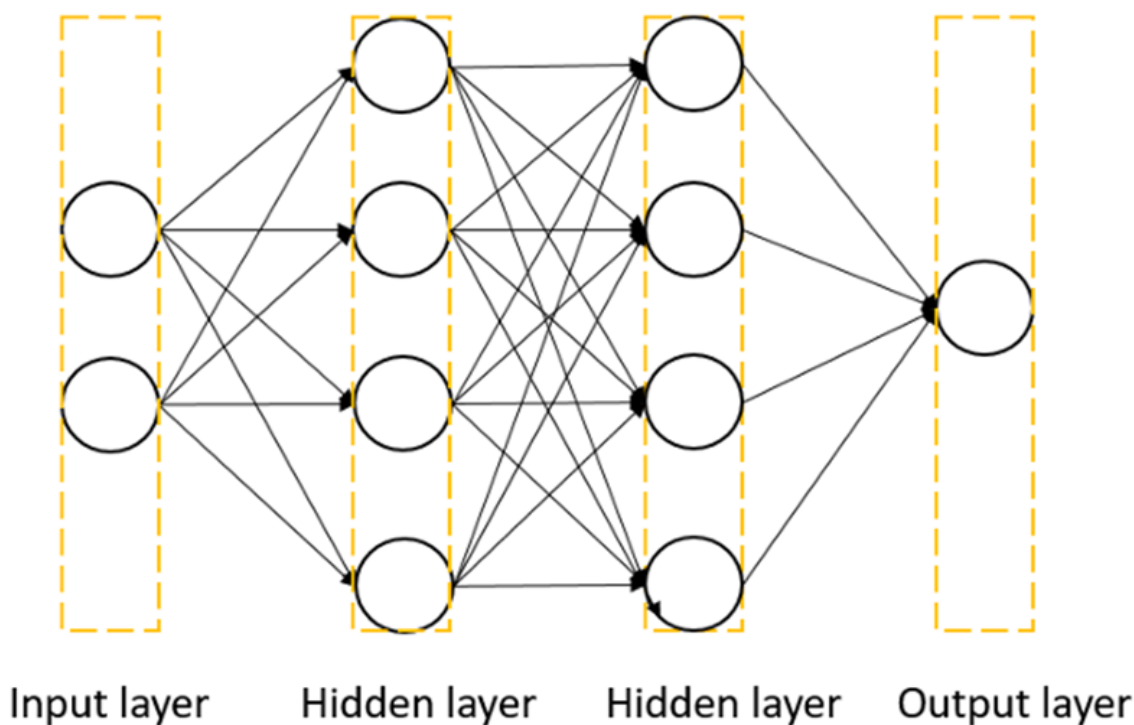


NYCU DLP Lab1 Report

1.Introduction

在本次作業裡實作了兩層hidden layer的Neural Network，並根據python standard library與numpy實做出模型的forward,backpropagation以及不同的activation function和optimizer，利用助教給的Linear Data和XOR Data 測試模型在不同的條件下，表現性能為如何



2.Experiments setup

A. Sigmoid function

透過sigmoid function來作為activation function，讓model能學習non-linear 的行為

```
def Sigmoid(x):
    return 1/ (1.0+np.exp(-x))
def Derivative_Sigmoid(x):
    return x * (1 - x)
```

B. Neural network

- one input layer with two neurons
- two hidden layer with ten neurons
- one output layer with one neurons
- default learning rate = 0.1
- default activate function is Sigmoid function
- default optimizer is SGD
- implement clipping gradient to avoid gradient explosion
- decay is for momentum

```
class Model:
    def __init__(self, input_size=2, hidden1_size=10, hidden2_size=10,
                  learning_rate = 0.1 , activate = "Sigmoid", optimizer = "SGD"):
        # init
        self.input_size = input_size
        self.hidden1_size = hidden1_size
        self.hidden2_size = hidden2_size
        self.output_size = output_size
        self.activate = activate
        self.learning_rate = learning_rate
        self.optimizer = optimizer

        #weight bias setting
        self.W1 = np.random.randn(input_size, hidden1_size)
        self.W2 = np.random.randn(hidden1_size, hidden2_size)
        self.W3 = np.random.randn(hidden2_size, output_size)
```

```

self.y=()

self.b1 = np.random.randn(1,hidden1_size)/100
self.b2 = np.random.randn(1,hidden2_size)/100
self.b3 = np.random.randn(1,output_size)/100

# Define hyperparameters for Momentum optimizer
self.v_w1 = 0 # Velocity for W1
self.v_w2 = 0 # Velocity for W2
self.v_w3 = 0 # Velocity for W3
self.v_b1 = 0 # Velocity for b1
self.v_b2 = 0 # Velocity for b2
self.v_b3 = 0 # Velocity for b3
self.decay =decay

def forward(self , x)      :
    self.x = x
    #first layer

    self.hidden1 = np.dot(x,self.W1) + self.b1

    if(self.activate == "Sigmoid") :
        self.hidden1 = Sigmoid(self.hidden1)
    elif(self.activate == "ReLu") :
        self.hidden1 = ReLu(self.hidden1)
    elif(self.activate == "tanh") :
        self.hidden1 = tanh(self.hidden1)
    elif self.activate == "None":
        pass

    #second layer
    self.hidden2 = np.dot(self.hidden1,self.W2) + self.b2

    if(self.activate == "Sigmoid") :

```

```

        self.hidden2 = Sigmoid(self.hidden2)
    elif(self.activate == "ReLu") :
        self.hidden2 = ReLu(self.hidden2)
    elif(self.activate == "tanh") :
        self.hidden2 = tanh(self.hidden2)
    elif self.activate == "None":
        pass

#output layer

self.y = np.dot(self.hidden2,self.W3) + self.b3

if(self.activate == "Sigmoid") :
    self.y = Sigmoid(self.y)
elif(self.activate == "ReLu") :
    self.y = ReLu(self.y)
elif(self.activate == "tanh") :
    self.y = tanh(self.y)
elif self.activate == "None":
    pass

return self.y

def clip_gradients(self, grad, threshold=1.0):
    grad = np.clip(grad, -threshold, threshold)
    return grad

def back_propagation(self,x,y_gt):
    #update w3 b3
    y_error =(self.y - y_gt)
    # error 乘 activation function的導數
    # 算gradient
    if(self.activate == "Sigmoid") :
        grad = y_error * Derivative_Sigmoid(self.y)
    elif(self.activate == "ReLu") :

```

```

        grad = y_error * Derivative_ReLu(self.y)
    elif(self.activate == "tanh") :
        grad = y_error * Derivative_tanh(self.y)
    elif self.activate == "None":
        grad=y_error

grad = self.clip_gradients(grad)

if(self.optimizer == "SGD"):
    #update
    self.W3 -= self.learning_rate * np.dot(self.hidden2, grad)
    self.b3 -= self.learning_rate * np.sum(grad,axis=0)
elif(self.optimizer == "Momentum"):
    self.v_W3 = self.decay *self.v_W3 + self.learning_rate * grad
    self.v_b3 =self.decay *self.v_b3 + self.learning_rate * np.sum(grad,axis=0)
    self.W3 -= self.v_W3
    self.b3 -= self.v_b3

    #update w2 b2
hidden2_error = np.dot(grad,self.W3.T)
if(self.activate == "Sigmoid") :
    grad = hidden2_error * Derivative_Sigmoid(self.hidden2)
elif(self.activate == "ReLu") :
    grad = hidden2_error * Derivative_ReLu(self.hidden2)
elif(self.activate == "tanh") :
    grad = hidden2_error * Derivative_tanh(self.hidden2)
elif self.activate == "None":
    grad=hidden2_error

grad = self.clip_gradients(grad)

#update
if(self.optimizer == "SGD"):
    self.W2 -= self.learning_rate * np.dot(self.hidden1, grad)
    self.b2 -= self.learning_rate * np.sum(grad,axis=0)
elif(self.optimizer == "Momentum"):

```

```

        self.v_W2 = self.decay * self.v_W2 + self.learning_rate * grad
        self.v_b2 = self.decay * self.v_b2 + self.learning_rate * grad
        self.W2 -= self.v_W2
        self.b2 -= self.v_b2
    #update w1 b1
    hidden1_error = np.dot(grad, self.W2.T)
    if(self.activate == "Sigmoid") :
        grad = hidden1_error * Derivative_Sigmoid(self.hidden1_output)
    elif(self.activate == "ReLu") :
        grad = hidden1_error * Derivative_ReLu(self.hidden1_output)
    elif(self.activate == "tanh") :
        grad = hidden1_error * Derivative_tanh(self.hidden1_output)
    elif self.activate == "None":
        grad = hidden1_error

    grad = self.clip_gradients(grad)

    #update
    if(self.optimizer == "SGD"):
        self.W1 -= self.learning_rate * np.dot(x.T, grad)
        self.b1 -= self.learning_rate * np.sum(grad, axis=0)
    elif(self.optimizer == "Momentum"):
        self.v_W1 = self.decay * self.v_W1 + self.learning_rate * grad
        self.v_b1 = self.decay * self.v_b1 + self.learning_rate * grad
        self.W1 -= self.v_W1
        self.b1 -= self.v_b1

    return np.sum ((self.y-y_gt) ** 2)/self.y.shape[0]

```

C. Backpropagation

Use chain rule and SGD to reduce the error between ground truth and prediction, also implement the Momentum.

```

def back_propagation(self,x,y_gt):
    #update w3 b3
    y_error =(self.y - y_gt)
    # error 乘 activation function的導數
    # 算gradient
    if(self.activate == "Sigmoid") :
        grad = y_error * Derivative_Sigmoid(self.y)
    elif(self.activate == "ReLu") :
        grad = y_error * Derivative_ReLu(self.y)
    elif(self.activate == "tanh") :
        grad = y_error * Derivative_tanh(self.y)
    elif self.activate == "None":
        grad=y_error

    grad = self.clip_gradients(grad)

    if(self.optimizer == "SGD"):
        #update
        self.W3 -= self.learning_rate * np.dot(self.hidden2, grad)
        self.b3 -= self.learning_rate * np.sum(grad,axis=0)
    elif(self.optimizer == "Momentum"):
        self.v_W3 = self.decay *self.v_W3 + self.learning_rate * grad
        self.v_b3 =self.decay *self.v_b3 + self.learning_rate * np.sum(grad,axis=0)
        self.W3 -= self.v_W3
        self.b3 -= self.v_b3

    #update w2 b2
    hidden2_error = np.dot(grad,self.W3.T)
    if(self.activate == "Sigmoid") :
        grad = hidden2_error * Derivative_Sigmoid(self.hidden2)
    elif(self.activate == "ReLu") :
        grad = hidden2_error * Derivative_ReLu(self.hidden2)
    elif(self.activate == "tanh") :
        grad = hidden2_error * Derivative_tanh(self.hidden2)
    elif self.activate == "None":

```

```

        grad=hidden2_error

grad = self.clip_gradients(grad)

#update
if(self.optimizer == "SGD"):
    self.W2 -= self.learning_rate * np.dot(self.hidden1, grad)
    self.b2 -= self.learning_rate * np.sum(grad,axis=0)
elif(self.optimizer == "Momentum"):
    self.v_W2 = self.decay *self.v_W2 + self.learning_rate * grad
    self.v_b2 =self.decay *self.v_b2 + self.learning_rate * np.sum(grad,axis=0)
    self.W2 -= self.v_W2
    self.b2 -= self.v_b2
#update w1 b1
hidden1_error = np.dot(grad,self.W2.T)
if(self.activate == "Sigmoid") :
    grad = hidden1_error * Derivative_Sigmoid(self.hidden1)
elif(self.activate == "ReLu") :
    grad = hidden1_error * Derivative_ReLu(self.hidden1)
elif(self.activate == "tanh") :
    grad = hidden1_error* Derivative_tanh(self.hidden1)
elif self.activate == "None":
    grad = hidden1_error

grad = self.clip_gradients(grad)

#update
if(self.optimizer == "SGD"):
    self.W1 -= self.learning_rate * np.dot(x.T, grad)
    self.b1 -= self.learning_rate * np.sum(grad,axis=0)
elif(self.optimizer == "Momentum"):
    self.v_W1 = self.decay *self.v_W1 + self.learning_rate * grad
    self.v_b1 =self.decay *self.v_b1 + self.learning_rate * np.sum(grad,axis=0)
    self.W1 -= self.v_W1
    self.b1 -= self.v_b1

```

```
return np.sum ((self.y-y_gt) ** 2)/self.y.shape[0]
```

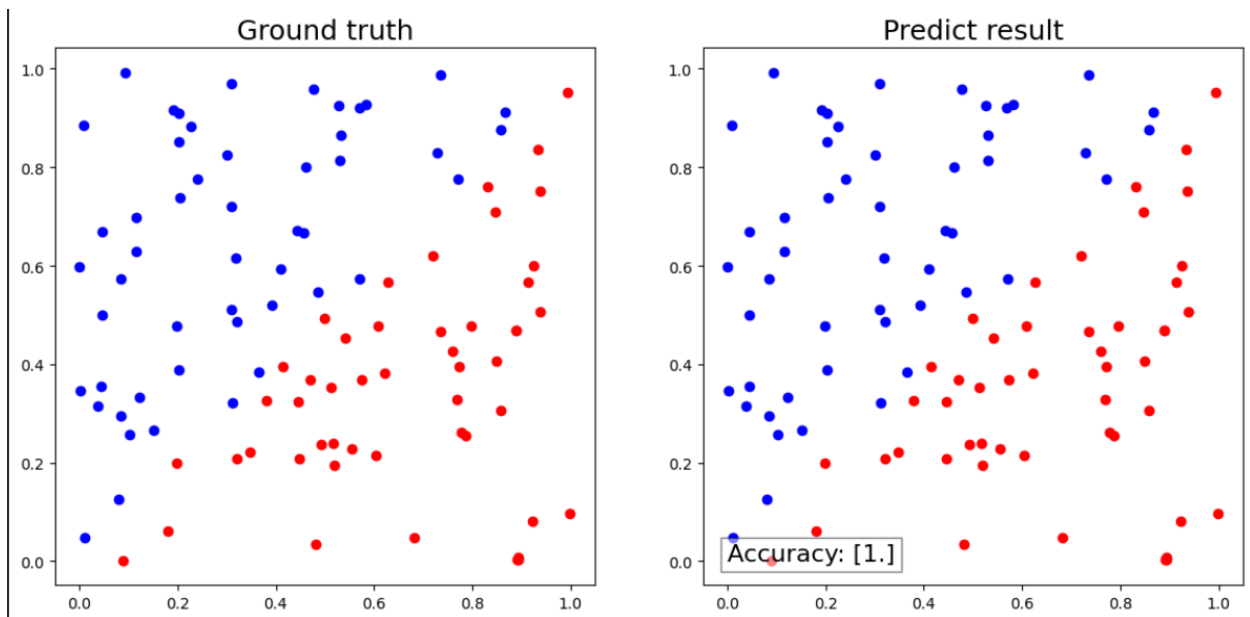
3. Result of your testing

epoch設10W

A. Screenshot and comparison figure

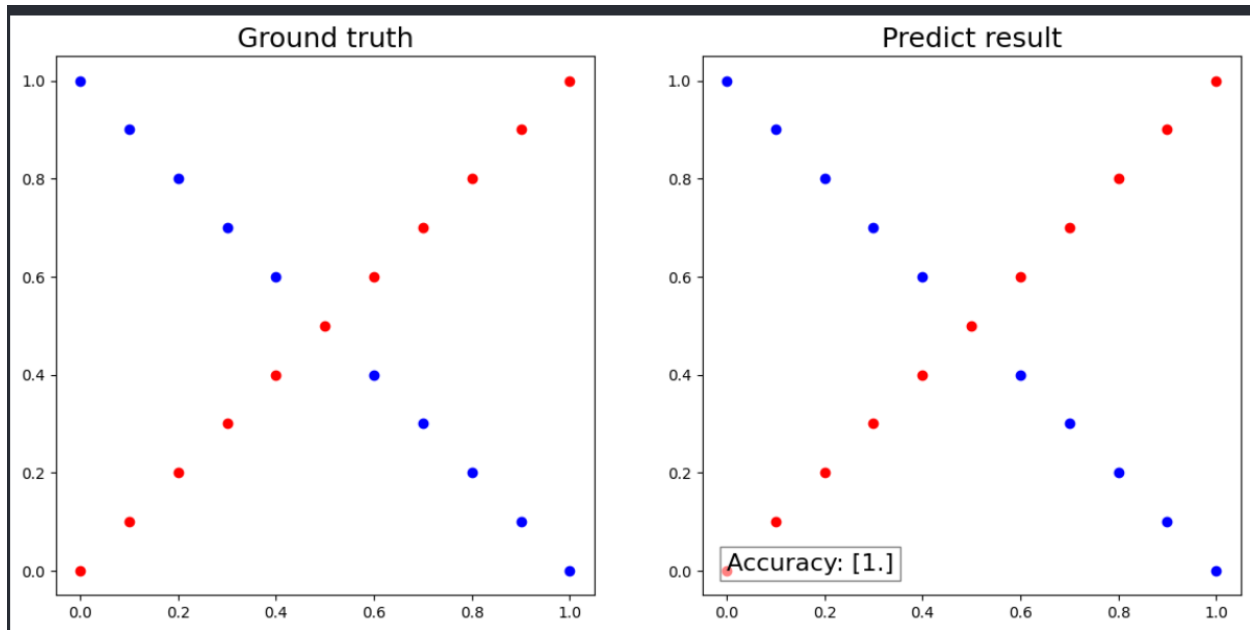
- Linear Data

```
----- Training -----  
epoch: 10000 loss: 0.0001003926041716694  
epoch: 20000 loss: 3.6007967464966466e-05  
epoch: 30000 loss: 2.049456461213936e-05  
epoch: 40000 loss: 1.3927213026789106e-05  
epoch: 50000 loss: 1.0390139177806662e-05  
epoch: 60000 loss: 8.209559608530463e-06  
epoch: 70000 loss: 6.743455046575347e-06  
epoch: 80000 loss: 5.696310842264765e-06  
epoch: 90000 loss: 4.914338643239946e-06  
epoch: 100000 loss: 4.31008132311798e-06
```



- XOR Data

```
----- Training -----  
epoch: 10000 loss: 0.00019915823941982173  
epoch: 20000 loss: 6.639028003611319e-05  
epoch: 30000 loss: 3.753856281065372e-05  
epoch: 40000 loss: 2.5557594621623407e-05  
epoch: 50000 loss: 1.9134101904542703e-05  
epoch: 60000 loss: 1.5174691486826007e-05  
epoch: 70000 loss: 1.2508779321532911e-05  
epoch: 80000 loss: 1.0600805401540442e-05  
epoch: 90000 loss: 9.172795795676663e-06  
epoch: 100000 loss: 8.066813891347705e-06
```



B. Show the accuracy of your prediction

- Linear Data

```
----- Testing -----
[[3.75110381e-08]
 [3.68588328e-08]
 [7.90418556e-06]
 [2.08215855e-08]
 [9.99999999e-01]
 [9.99999996e-01]
 [9.99894192e-01]
 [1.87584698e-08]
 [9.99999915e-01]
 [9.99999999e-01]
 [9.99999998e-01]
 [1.15070735e-06]
 [9.99999998e-01]
 [2.66478001e-08]
 [1.04329403e-02]
 [9.99999998e-01]
 [4.01170734e-08]]
```

[2.33291181e-08]
[9.99999999e-01]
[9.99999999e-01]
[3.70036108e-08]
[2.36711179e-08]
[9.99999999e-01]
[1.90757772e-08]
[9.99999998e-01]
[9.99999884e-01]
[9.99999998e-01]
[9.99999997e-01]
[2.64376032e-05]
[9.99999994e-01]
[1.94315936e-08]
[9.99999996e-01]
[1.80947311e-08]
[2.27796091e-08]
[8.39367037e-08]
[2.04853204e-08]
[2.26535810e-08]
[9.99999996e-01]
[2.95691175e-08]
[9.99999999e-01]
[1.83101515e-08]
[1.99947676e-08]
[9.99999999e-01]
[2.40539199e-08]
[9.99999992e-01]
[2.62376610e-07]
[2.39621037e-08]
[4.88682280e-03]
[1.85237257e-08]
[3.30910127e-07]
[4.22218665e-06]
[2.18012010e-08]
[9.99999999e-01]

[9.99999962e-01]
[9.99999998e-01]
[2.82674180e-08]
[9.99999935e-01]
[9.99999999e-01]
[1.93703478e-08]
[9.99999998e-01]
[9.99999999e-01]
[9.99999637e-01]
[1.87487845e-08]
[1.90804217e-08]
[1.96861190e-08]
[9.99999999e-01]
[9.99999983e-01]
[2.08430623e-07]
[8.85452400e-03]
[9.99999999e-01]
[9.99999999e-01]
[2.82527363e-07]
[9.99999999e-01]
[2.27048921e-08]
[1.93186012e-08]
[1.90504832e-08]
[2.58141513e-08]
[6.91679494e-08]
[1.88641438e-08]
[2.77949073e-06]
[9.99999998e-01]
[1.92773565e-08]
[9.85517345e-01]
[9.99999991e-01]
[9.99999996e-01]
[9.99999999e-01]
[2.03019981e-08]
[9.99999999e-01]
[1.83711461e-08]

```
[2.51921693e-08]
[2.46424624e-08]
[9.99999999e-01]
[2.92379225e-08]
[9.99999999e-01]
[9.99996046e-01]
[5.60636676e-06]
[9.99999998e-01]
[9.99999221e-01]
[9.99999999e-01]
[3.18018670e-03]]
```

-----Iteration-----

```
Iter:1 |      Ground truth : [0] |    prediction : [3.75110381e-01]
Iter:2 |      Ground truth : [0] |    prediction : [3.68588328e-01]
Iter:3 |      Ground truth : [0] |    prediction : [7.90418556e-01]
Iter:4 |      Ground truth : [0] |    prediction : [2.08215855e-01]
Iter:5 |      Ground truth : [1] |    prediction : [1.]
Iter:6 |      Ground truth : [1] |    prediction : [1.]
Iter:7 |      Ground truth : [1] |    prediction : [0.99989419]
Iter:8 |      Ground truth : [0] |    prediction : [1.87584698e-01]
Iter:9 |      Ground truth : [1] |    prediction : [0.99999991]
Iter:10 |     Ground truth : [1] |    prediction : [1.]
Iter:11 |     Ground truth : [1] |    prediction : [1.]
Iter:12 |     Ground truth : [0] |    prediction : [1.15070735e-01]
Iter:13 |     Ground truth : [1] |    prediction : [1.]
Iter:14 |     Ground truth : [0] |    prediction : [2.66478001e-01]
Iter:15 |     Ground truth : [0] |    prediction : [0.01043294]
Iter:16 |     Ground truth : [1] |    prediction : [1.]
Iter:17 |     Ground truth : [0] |    prediction : [4.01170734e-01]
Iter:18 |     Ground truth : [0] |    prediction : [2.33291181e-01]
Iter:19 |     Ground truth : [1] |    prediction : [1.]
Iter:20 |     Ground truth : [1] |    prediction : [1.]
Iter:21 |     Ground truth : [0] |    prediction : [3.70036108e-01]
Iter:22 |     Ground truth : [0] |    prediction : [2.36711179e-01]
Iter:23 |     Ground truth : [1] |    prediction : [1.]
Iter:24 |     Ground truth : [0] |    prediction : [1.90757772e-01]
```

Iter:25		Ground truth : [1]		prediction : [1.]
Iter:26		Ground truth : [1]		prediction : [0.99999988]
Iter:27		Ground truth : [1]		prediction : [1.]
Iter:28		Ground truth : [1]		prediction : [1.]
Iter:29		Ground truth : [0]		prediction : [2.64376032e-
Iter:30		Ground truth : [1]		prediction : [0.99999999]
Iter:31		Ground truth : [0]		prediction : [1.94315936e-
Iter:32		Ground truth : [1]		prediction : [1.]
Iter:33		Ground truth : [0]		prediction : [1.80947311e-
Iter:34		Ground truth : [0]		prediction : [2.27796091e-
Iter:35		Ground truth : [0]		prediction : [8.39367037e-
Iter:36		Ground truth : [0]		prediction : [2.04853204e-
Iter:37		Ground truth : [0]		prediction : [2.2653581e-(
Iter:38		Ground truth : [1]		prediction : [1.]
Iter:39		Ground truth : [0]		prediction : [2.95691175e-
Iter:40		Ground truth : [1]		prediction : [1.]
Iter:41		Ground truth : [0]		prediction : [1.83101515e-
Iter:42		Ground truth : [0]		prediction : [1.99947676e-
Iter:43		Ground truth : [1]		prediction : [1.]
Iter:44		Ground truth : [0]		prediction : [2.40539199e-
Iter:45		Ground truth : [1]		prediction : [0.99999999]
Iter:46		Ground truth : [0]		prediction : [2.6237661e-(
Iter:47		Ground truth : [0]		prediction : [2.39621037e-
Iter:48		Ground truth : [0]		prediction : [0.00488682]
Iter:49		Ground truth : [0]		prediction : [1.85237257e-
Iter:50		Ground truth : [0]		prediction : [3.30910127e-
Iter:51		Ground truth : [0]		prediction : [4.22218665e-
Iter:52		Ground truth : [0]		prediction : [2.1801201e-(
Iter:53		Ground truth : [1]		prediction : [1.]
Iter:54		Ground truth : [1]		prediction : [0.99999996]
Iter:55		Ground truth : [1]		prediction : [1.]
Iter:56		Ground truth : [0]		prediction : [2.8267418e-(
Iter:57		Ground truth : [1]		prediction : [0.99999994]
Iter:58		Ground truth : [1]		prediction : [1.]
Iter:59		Ground truth : [0]		prediction : [1.93703478e-
Iter:60		Ground truth : [1]		prediction : [1.]

Iter:61		Ground truth : [1]		prediction : [1.]
Iter:62		Ground truth : [1]		prediction : [0.99999964]
Iter:63		Ground truth : [0]		prediction : [1.87487845e-
Iter:64		Ground truth : [0]		prediction : [1.90804217e-
Iter:65		Ground truth : [0]		prediction : [1.9686119e-(
Iter:66		Ground truth : [1]		prediction : [1.]
Iter:67		Ground truth : [1]		prediction : [0.99999998]
Iter:68		Ground truth : [0]		prediction : [2.08430623e-
Iter:69		Ground truth : [0]		prediction : [0.00885452]
Iter:70		Ground truth : [1]		prediction : [1.]
Iter:71		Ground truth : [1]		prediction : [1.]
Iter:72		Ground truth : [0]		prediction : [2.82527363e-
Iter:73		Ground truth : [1]		prediction : [1.]
Iter:74		Ground truth : [0]		prediction : [2.27048921e-
Iter:75		Ground truth : [0]		prediction : [1.93186012e-
Iter:76		Ground truth : [0]		prediction : [1.90504832e-
Iter:77		Ground truth : [0]		prediction : [2.58141513e-
Iter:78		Ground truth : [0]		prediction : [6.91679494e-
Iter:79		Ground truth : [0]		prediction : [1.88641438e-
Iter:80		Ground truth : [0]		prediction : [2.77949073e-
Iter:81		Ground truth : [1]		prediction : [1.]
Iter:82		Ground truth : [0]		prediction : [1.92773565e-
Iter:83		Ground truth : [1]		prediction : [0.98551735]
Iter:84		Ground truth : [1]		prediction : [0.99999999]
Iter:85		Ground truth : [1]		prediction : [1.]
Iter:86		Ground truth : [1]		prediction : [1.]
Iter:87		Ground truth : [0]		prediction : [2.03019981e-
Iter:88		Ground truth : [1]		prediction : [1.]
Iter:89		Ground truth : [0]		prediction : [1.83711461e-
Iter:90		Ground truth : [0]		prediction : [2.51921693e-
Iter:91		Ground truth : [0]		prediction : [2.46424624e-
Iter:92		Ground truth : [1]		prediction : [1.]
Iter:93		Ground truth : [0]		prediction : [2.92379225e-
Iter:94		Ground truth : [1]		prediction : [1.]
Iter:95		Ground truth : [1]		prediction : [0.99999605]
Iter:96		Ground truth : [0]		prediction : [5.60636676e-

```

Iter:97 |      Ground truth : [1] |    prediction : [1.]
Iter:98 |      Ground truth : [1] |    prediction : [0.99999922]
Iter:99 |      Ground truth : [1] |    prediction : [1.]
Iter:100 |      Ground truth : [0] |    prediction : [0.00318019]
-----accuracy-----
accuracy:  100.0 %

```

- XOR Data

```

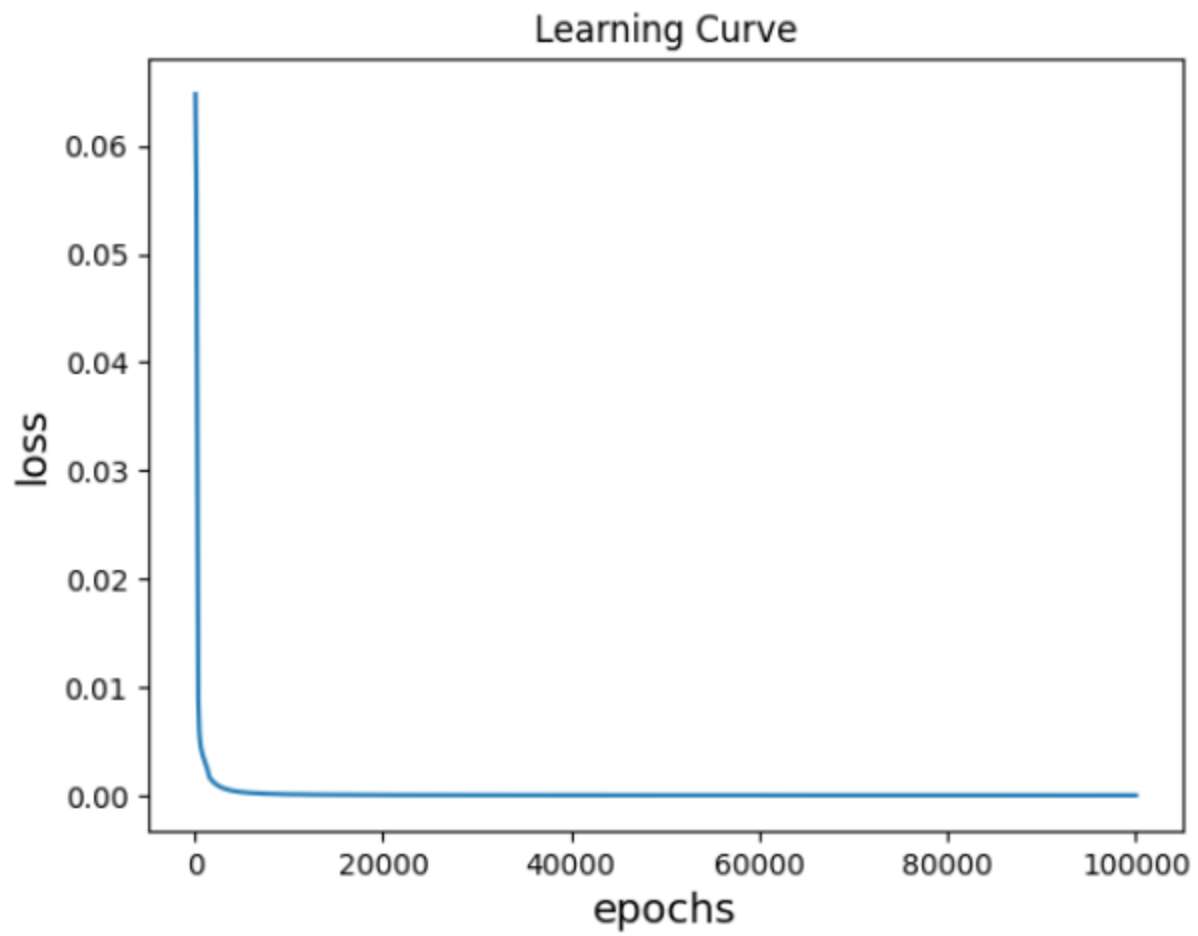
----- Testing -----
[[1.70527325e-03
 9.99999845e-01]
 [7.23660106e-04
 9.99999803e-01]
 [5.20309113e-04
 9.99999681e-01]
 [8.91155626e-04
 9.9998157e-01]
 [3.22045505e-03
 9.93212535e-01]
 [7.07568755e-03
 4.10702904e-03]
 [9.93679616e-01
 1.20412078e-03]
 [9.99997857e-01
 3.41791595e-04]
 [9.99999168e-01
 1.17958530e-04]
 [9.9998990e-01
 5.05447206e-05]
 [9.9998514e-01]]
-----Iteration-----
Iter:1 |      Ground truth : [0] |    prediction : [0.00170527]
Iter:2 |      Ground truth : [1] |    prediction : [0.99999985]
Iter:3 |      Ground truth : [0] |    prediction : [0.00072366]

```

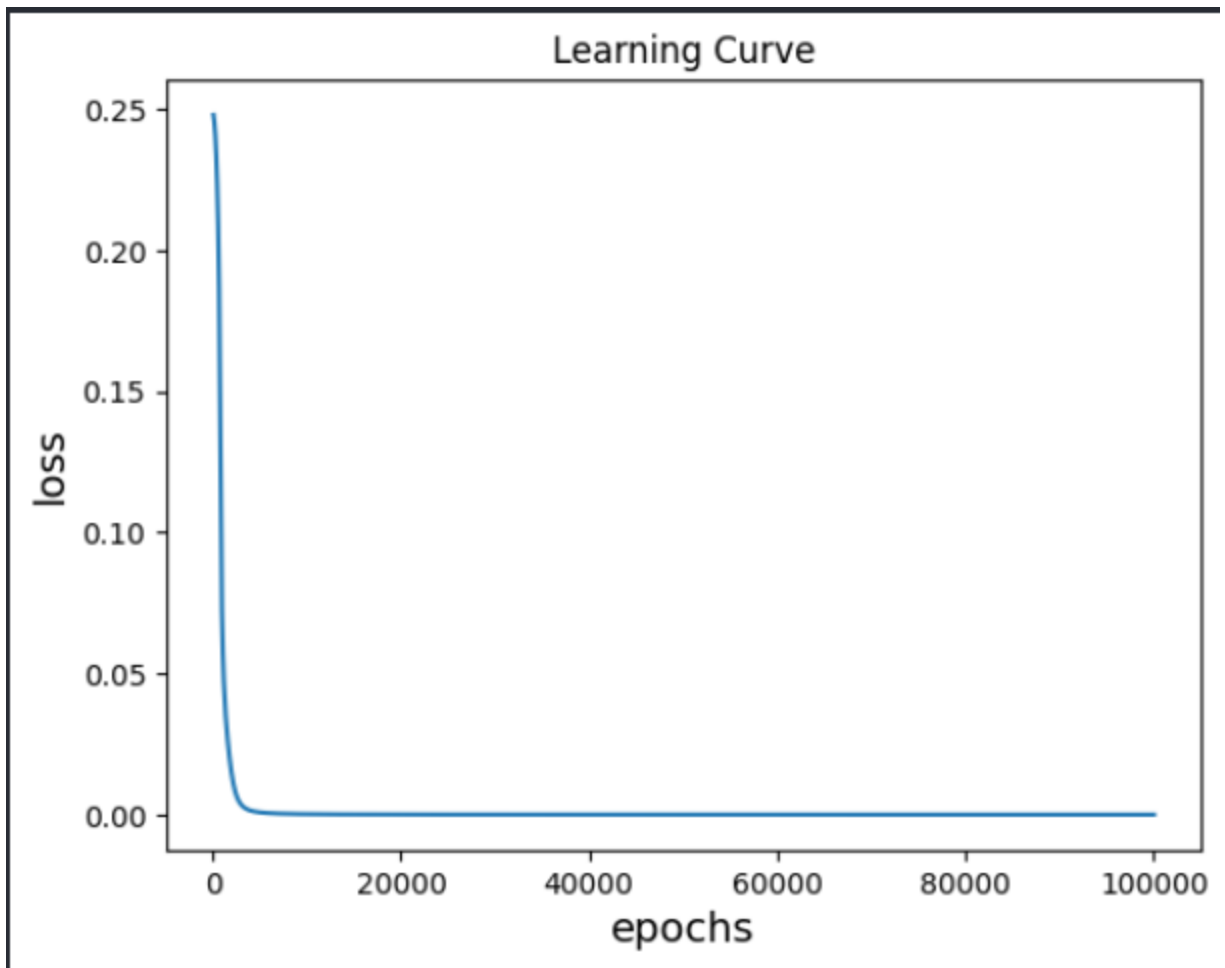
```
Iter:4 |      Ground truth : [1] |      prediction : [0.9999998]
Iter:5 |      Ground truth : [0] |      prediction : [0.00052031]
Iter:6 |      Ground truth : [1] |      prediction : [0.99999968]
Iter:7 |      Ground truth : [0] |      prediction : [0.00089116]
Iter:8 |      Ground truth : [1] |      prediction : [0.99999816]
Iter:9 |      Ground truth : [0] |      prediction : [0.00322046]
Iter:10 |     Ground truth : [1] |      prediction : [0.99321254]
Iter:11 |     Ground truth : [0] |      prediction : [0.00707569]
Iter:12 |     Ground truth : [0] |      prediction : [0.00410703]
Iter:13 |     Ground truth : [1] |      prediction : [0.99367962]
Iter:14 |     Ground truth : [0] |      prediction : [0.00120412]
Iter:15 |     Ground truth : [1] |      prediction : [0.99999786]
Iter:16 |     Ground truth : [0] |      prediction : [0.00034179]
Iter:17 |     Ground truth : [1] |      prediction : [0.99999917]
Iter:18 |     Ground truth : [0] |      prediction : [0.0001179]
Iter:19 |     Ground truth : [1] |      prediction : [0.99999899]
Iter:20 |     Ground truth : [0] |      prediction : [5.05447206e-
Iter:21 |     Ground truth : [1] |      prediction : [0.99999851]
-----accuracy-----
accuracy:  100.0 %
```

C. Learning curve (loss, epoch curve)

- Linear Data



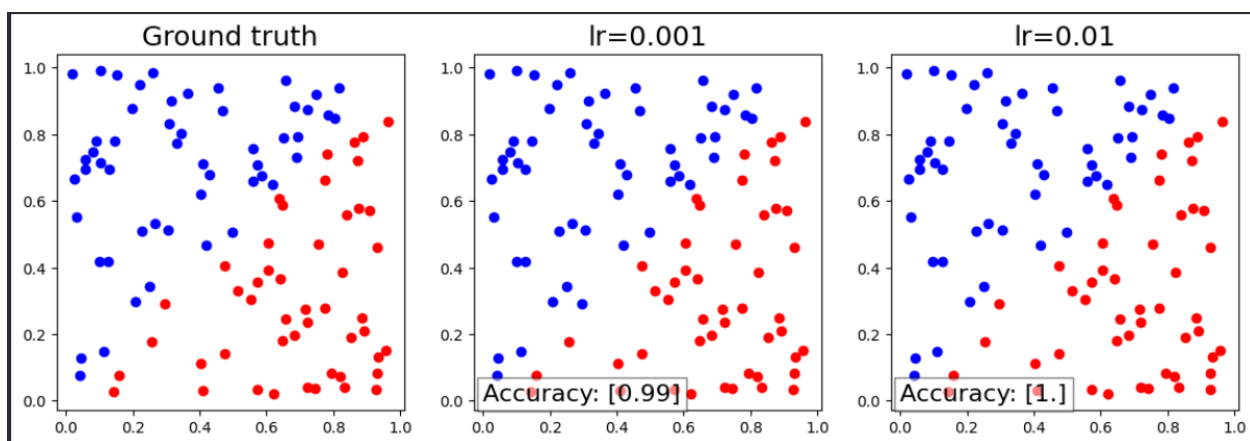
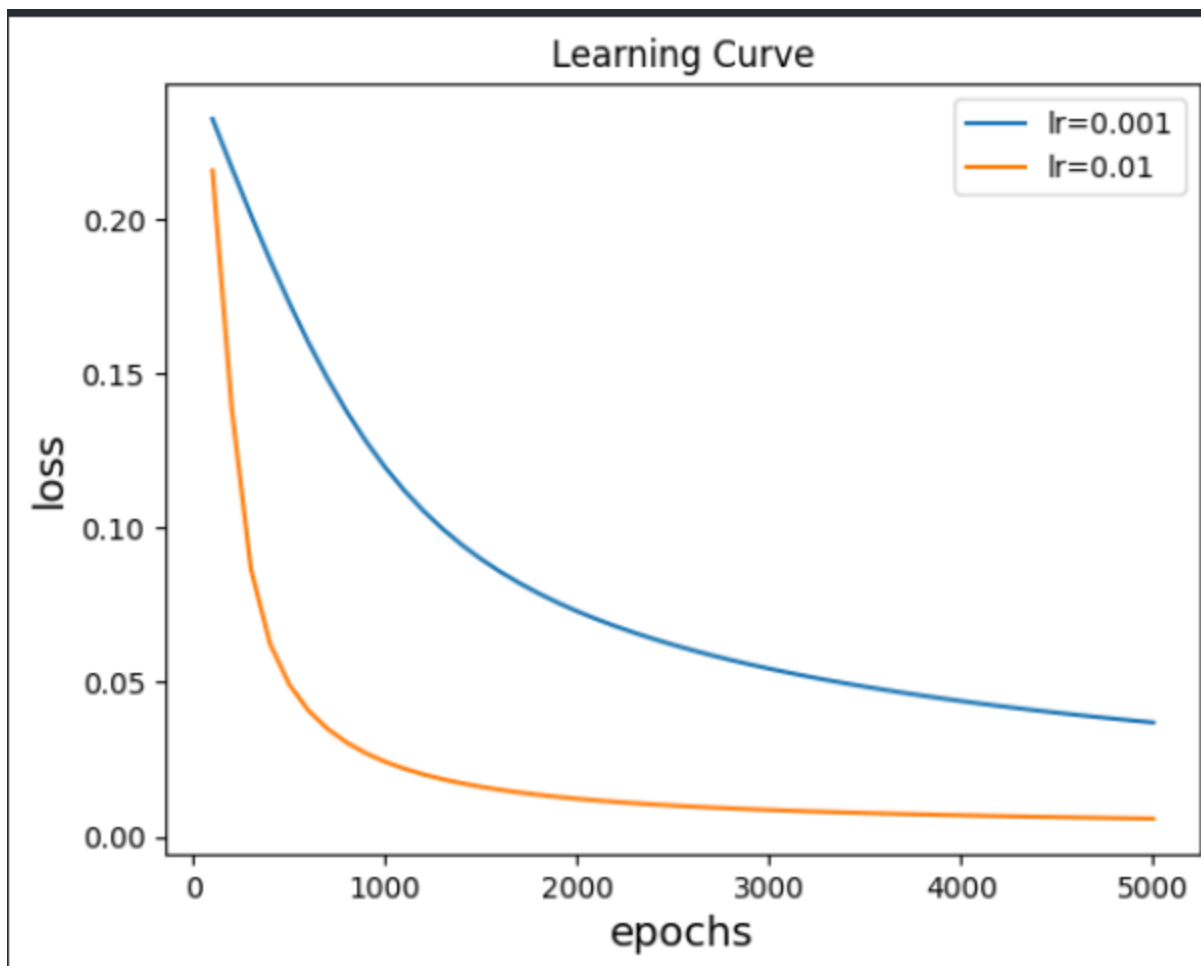
- XOR Data



4. Discussion

A. Try different learning rates

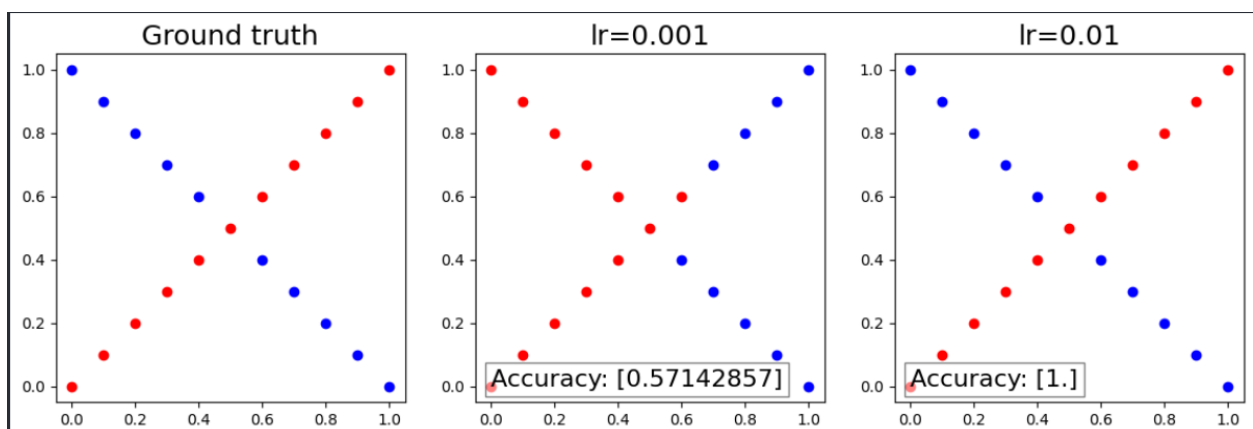
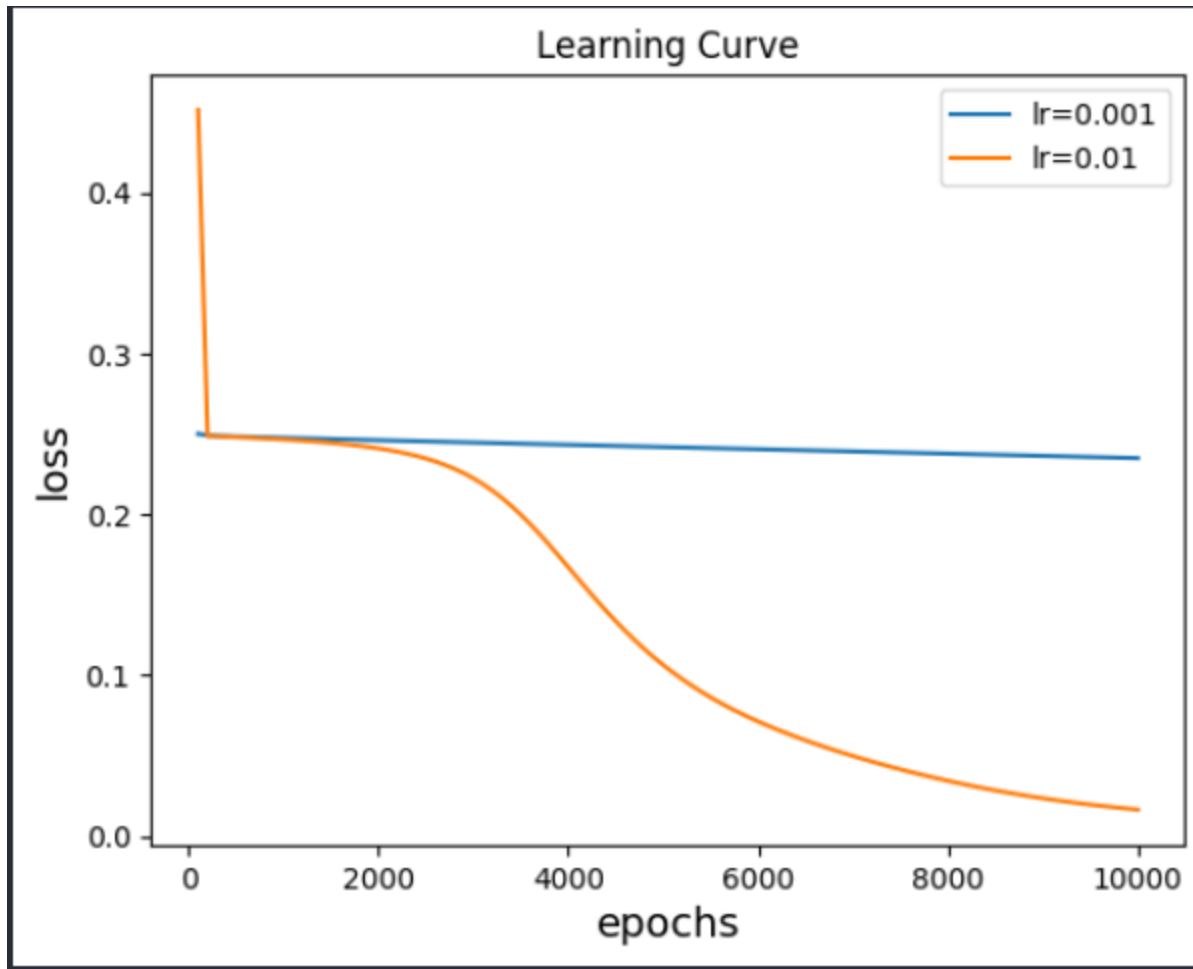
- Linear Data
 - epoch : 5000
 - model_1_lr : 0.001
 - model_2_lr : 0.01



可以看出當 $lr = 0.01$ 時，Loss下降比較快，在此Data Set上表現得比較好

- XOR Data

- epoch 調成10000，其他與上方相同



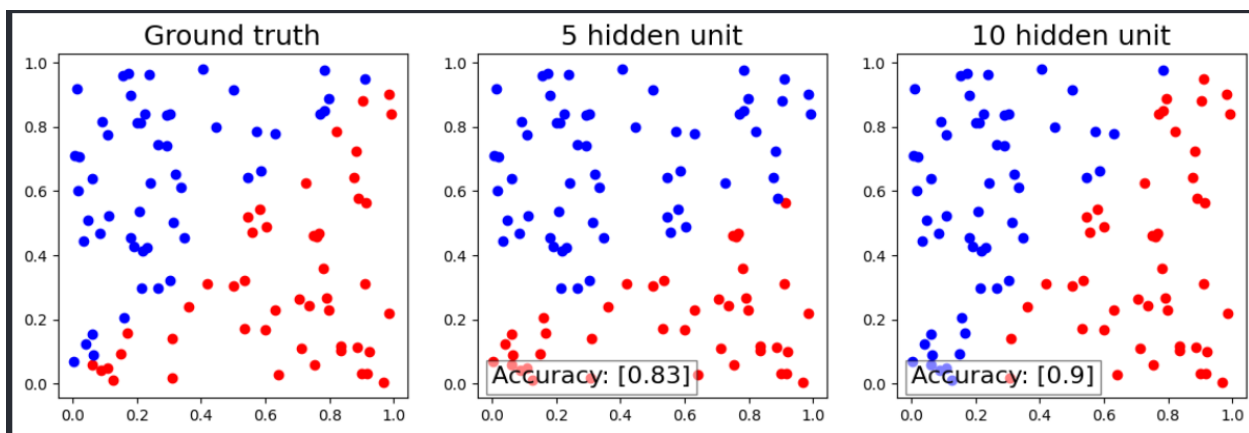
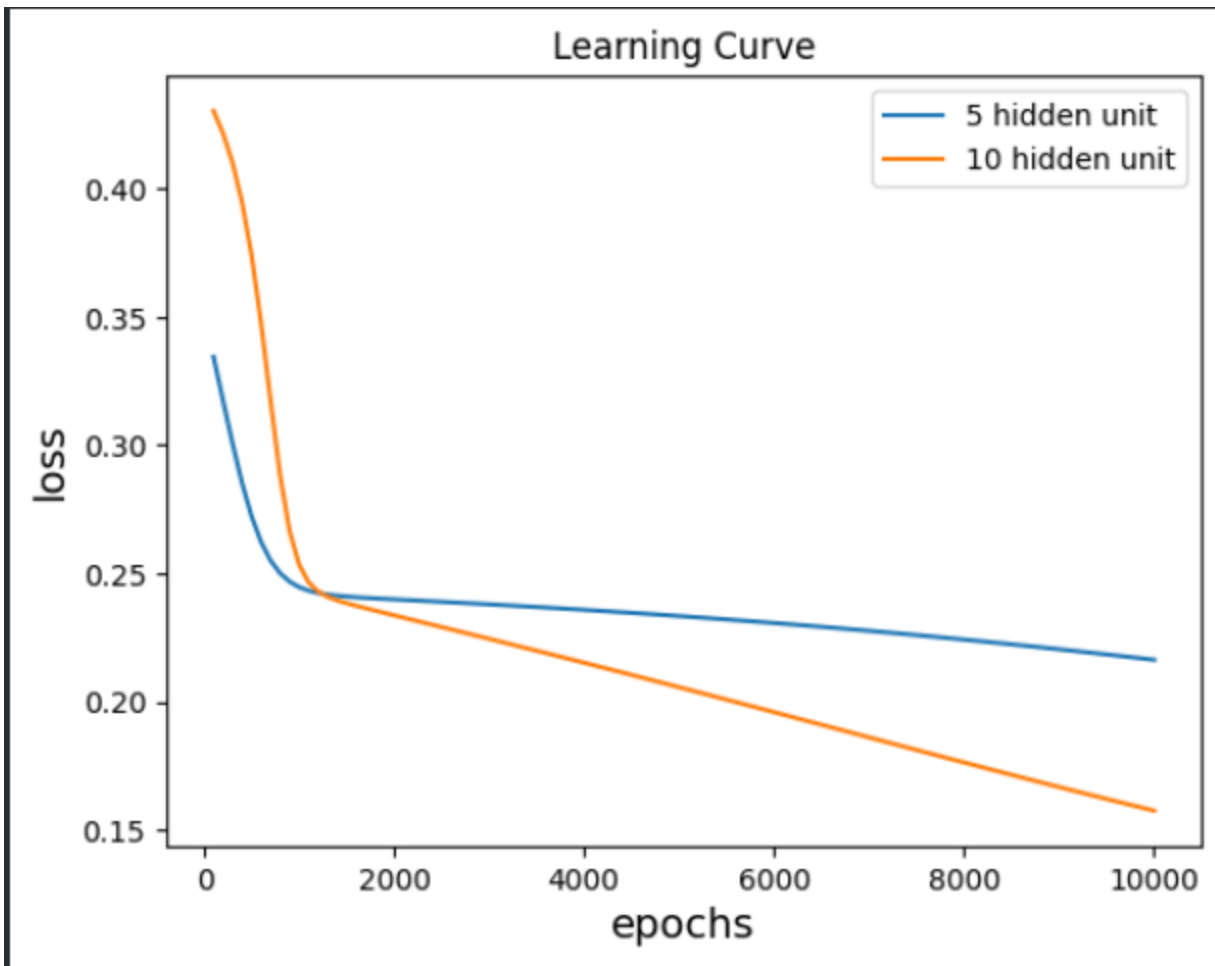


實驗結果也是 $lr=0.01$ 時表現比較好，在此Data Set尤其明顯，可以發現在此兩種Data Set越小 lr ，結果通常比較好
若epoch調高 $lr=0.001$ 也能達到100%，但這邊為了比較出實驗結果，選擇較低的epoch

B. Try different numbers of hidden units

- Linear Data

```
x,y =generate_linear()  
epochs = 10000  
model1 =Model(hidden1_size= 5,hidden2_size= 5,learning_rate= 1e-4)  
model2 = Model(hidden1_size= 10,hidden2_size= 10,learning_rate= 1e-4 )  
# training  
compare(x,y,model1,model2,"5 hidden unit","10 hidden unit",epochs)
```



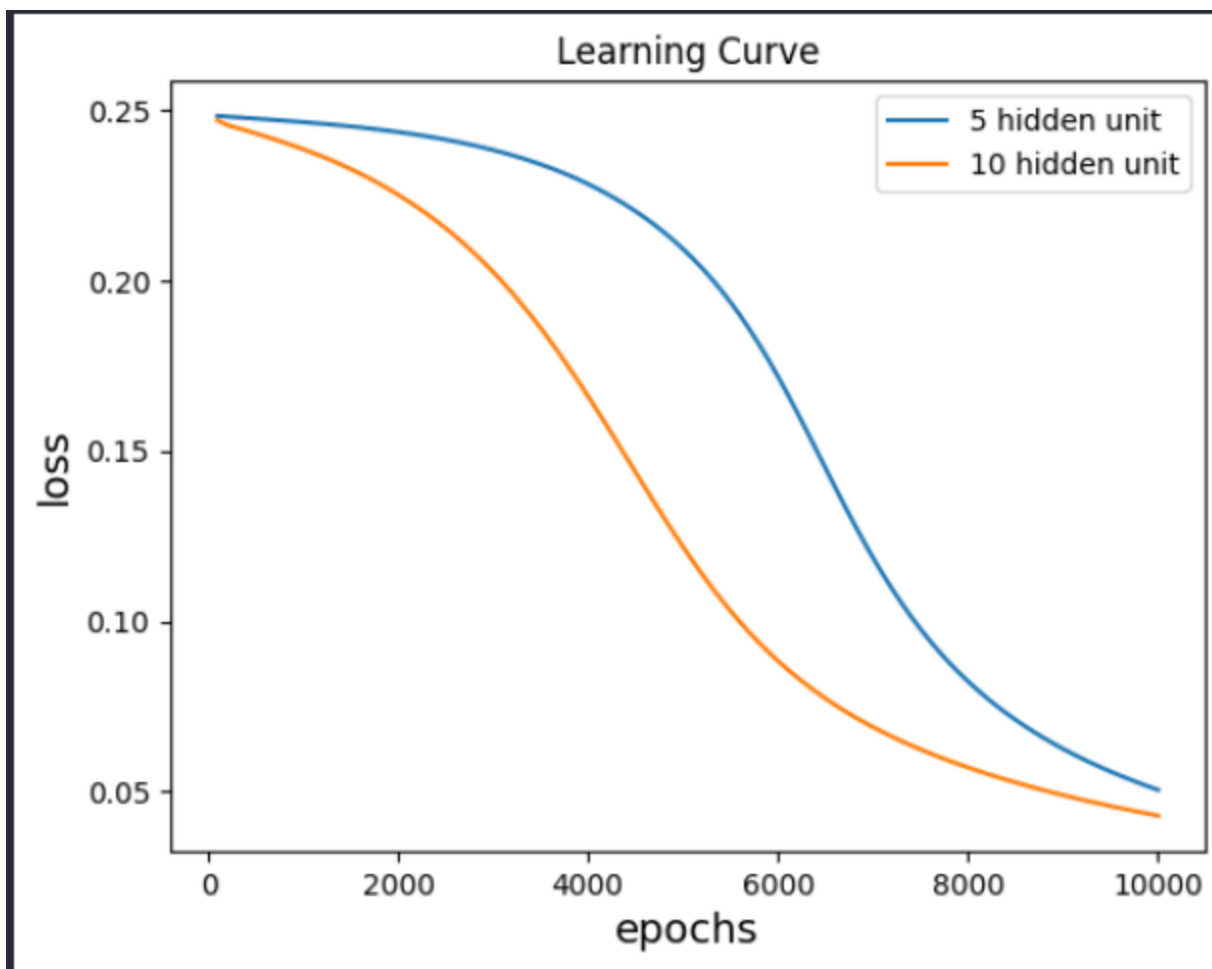
model在10 hidden-unit上 表現較佳，
若epoch調高 $lr=0.001$ 也能達到100% ，但這邊為了比較出實驗結果，選擇較低的epoch

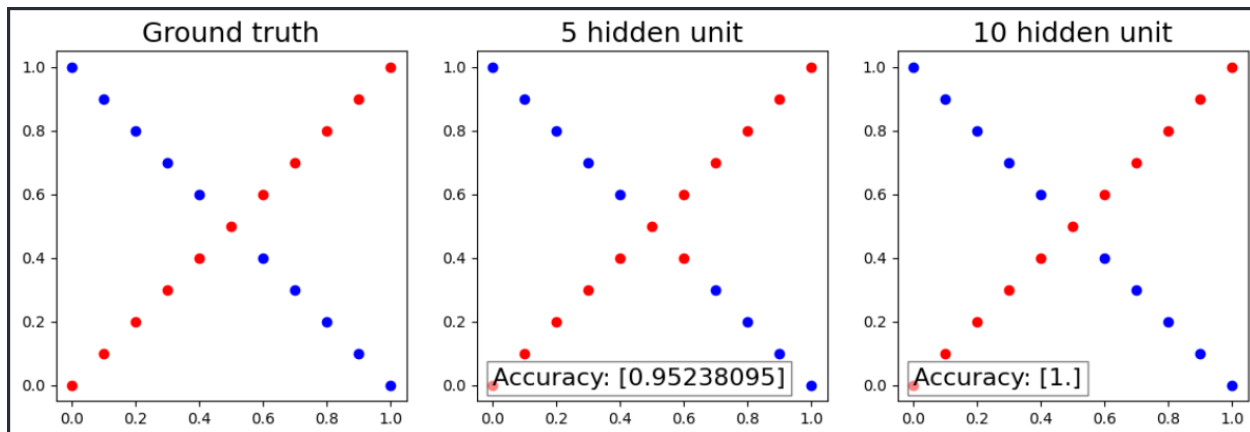
- XOR Data

```
x_xor,y_xor =generate_XOR_easy()

epochs = 10000

model1 =Model( hidden1_size= 5,hidden2_size=5,learning_rate= 1e-2)
model2 = Model(hidden1_size= 10,hidden2_size=10,learning_rate= 1e-2)
# training
compare(x_xor,y_xor,model1,model2,"5 hidden unit","10 hidden unit",epochs)
```





XOR Data Set則是 5 hidden unit表現略好一點，但調高epoch 或調低lr皆能達到100%accuracy

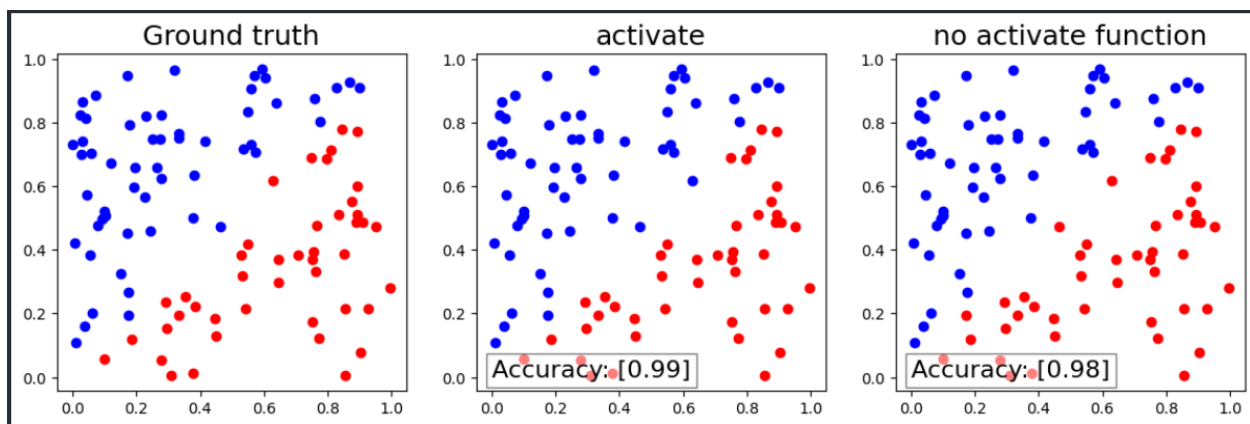
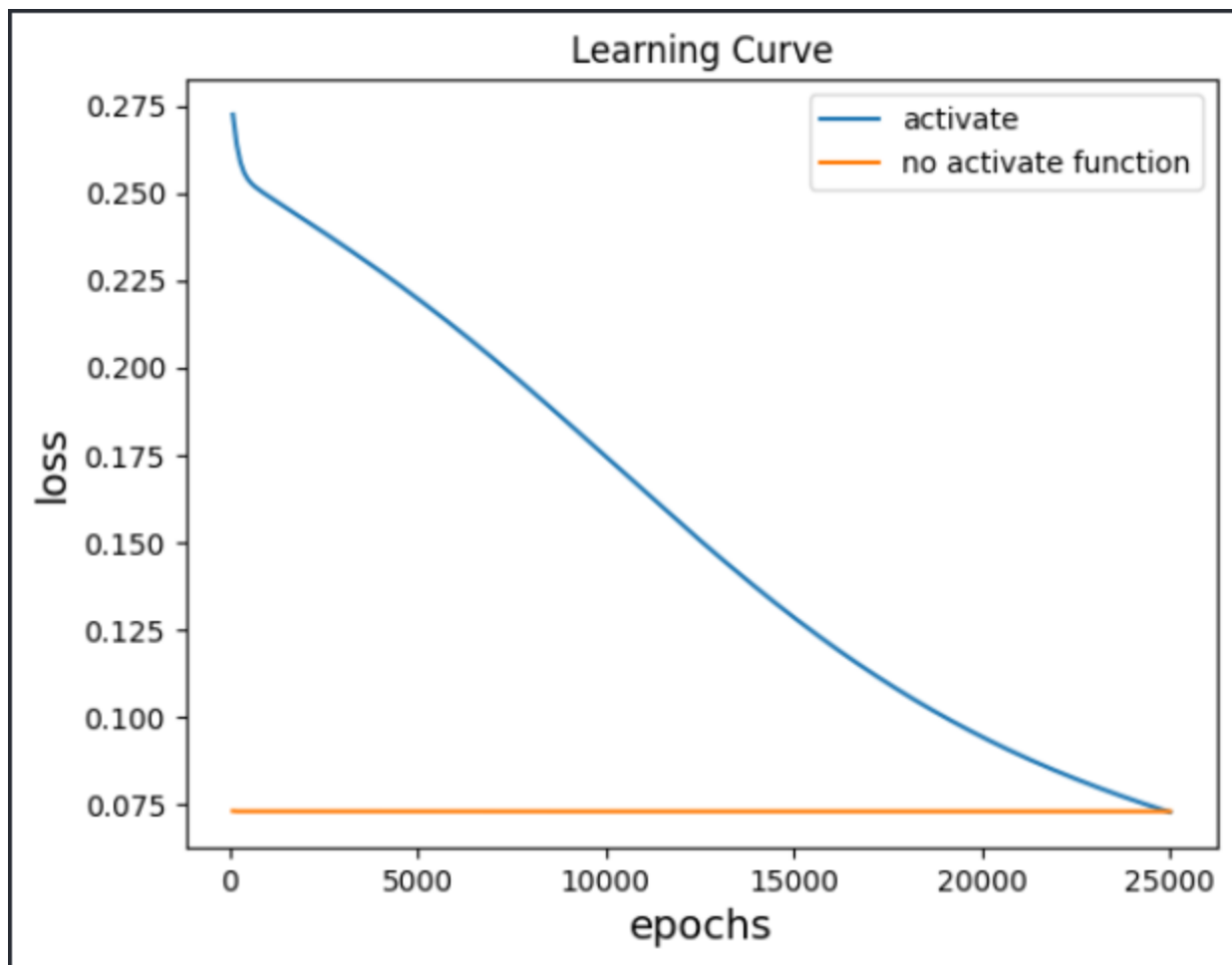
C. Try without activation functions

- Linear Data

```

• x,y =generate_linear()
epochs = 25000
model1 =Model(learning_rate= 1e-4)
model2 = Model(activate="None",learning_rate= 1e-4 )
# training
compare(x,y,model1,model2,"activate","no activate function",epochs)

```



可以發現在linear Data中有無activate function差異不大

- XOR Data

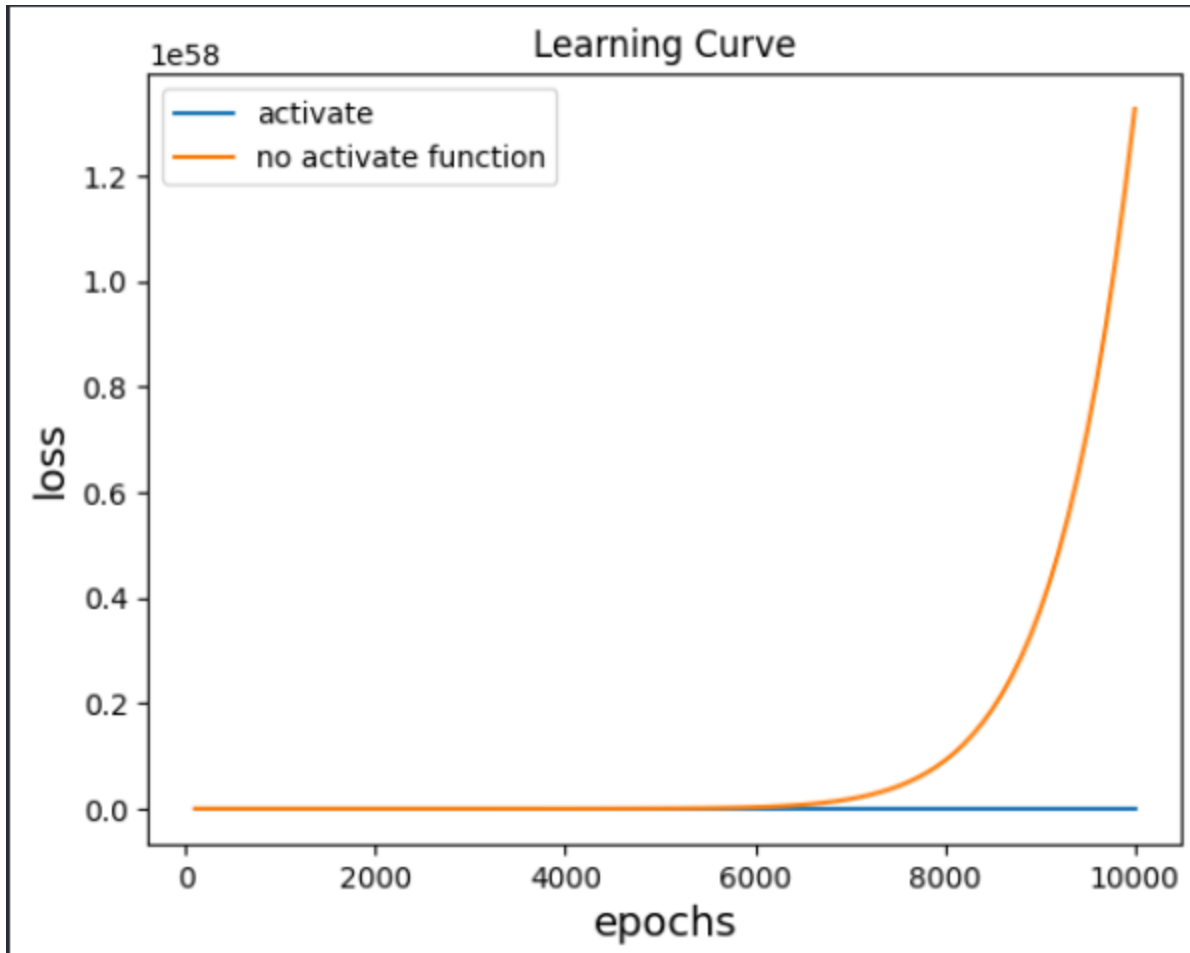
```

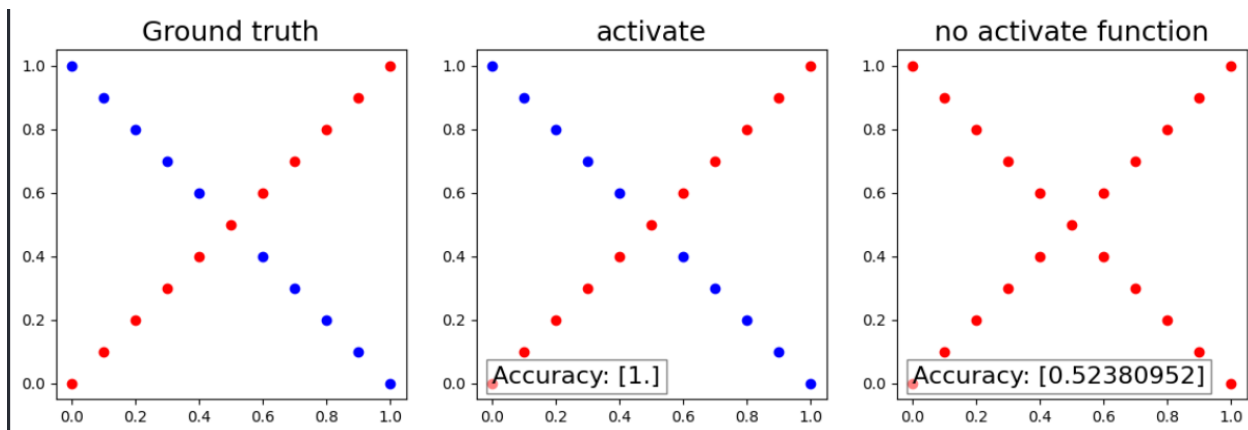
x_xor,y_xor =generate_XOR_easy()

epochs = 10000

model1 =Model( learning_rate= 1e-1)
model2 = Model(activate="None",learning_rate= 1e-1)
# training
compare(x_xor,y_xor,model1,model2,"activate","no activate function",epochs)

```





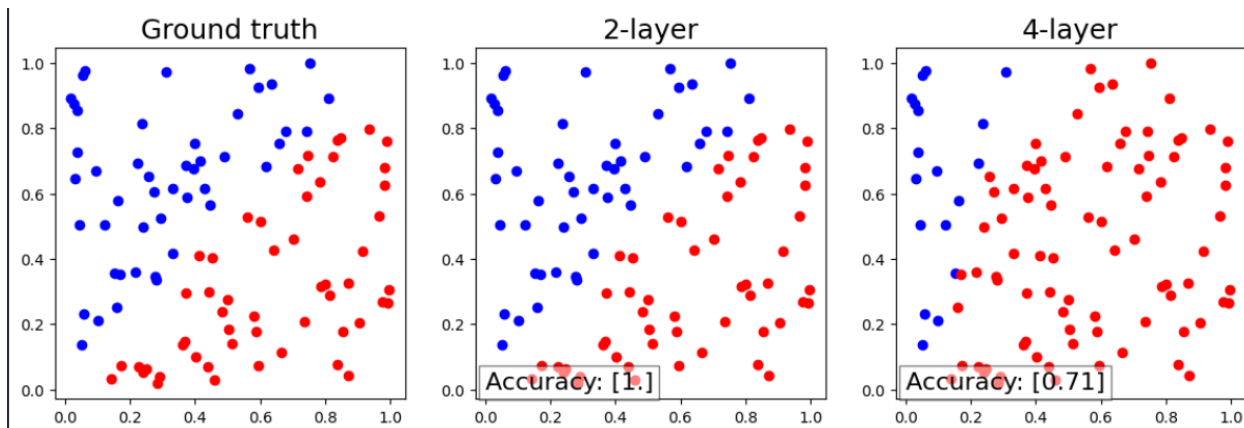
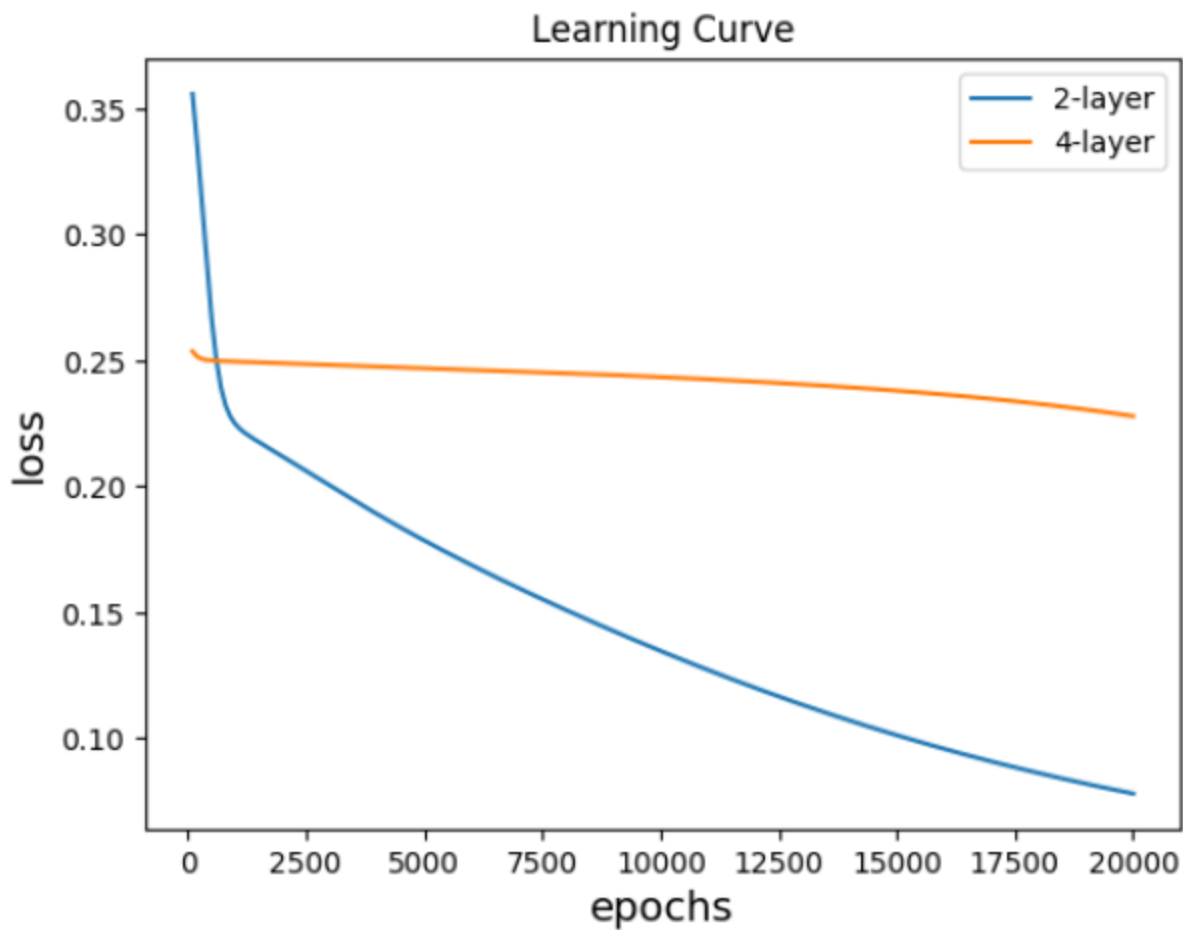
可以發現沒有activate function 在 non linear data set學習的比教爛

D. Different hidden layer (2 vs 4)

- Linear

```
x,y =generate_linear()
epochs = 20000
model1 = Model(learning_rate=1e-4)
model2 = Model_4layer(learning_rate=1e-4)

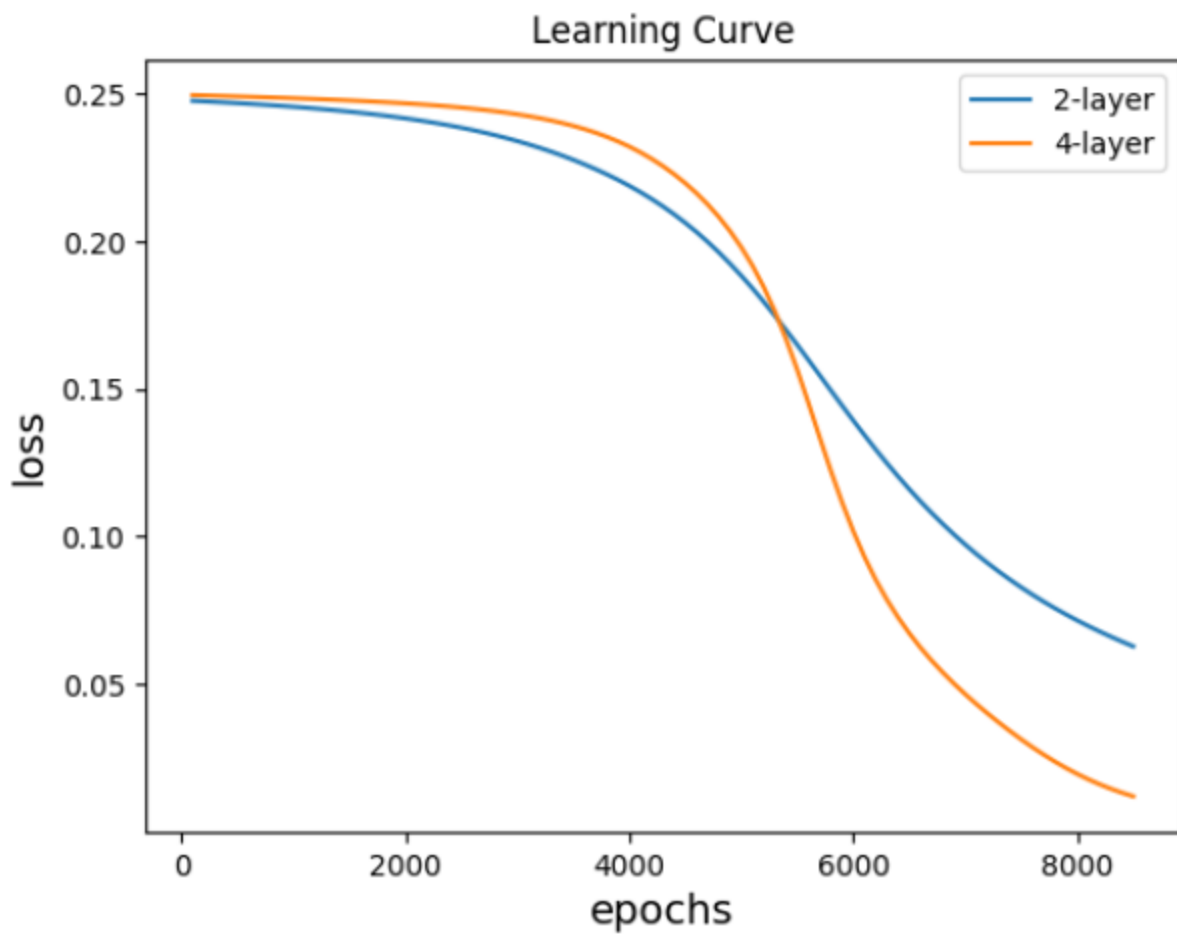
compare(x,y,model1,model2,"2-layer","4-layer",epochs)
```

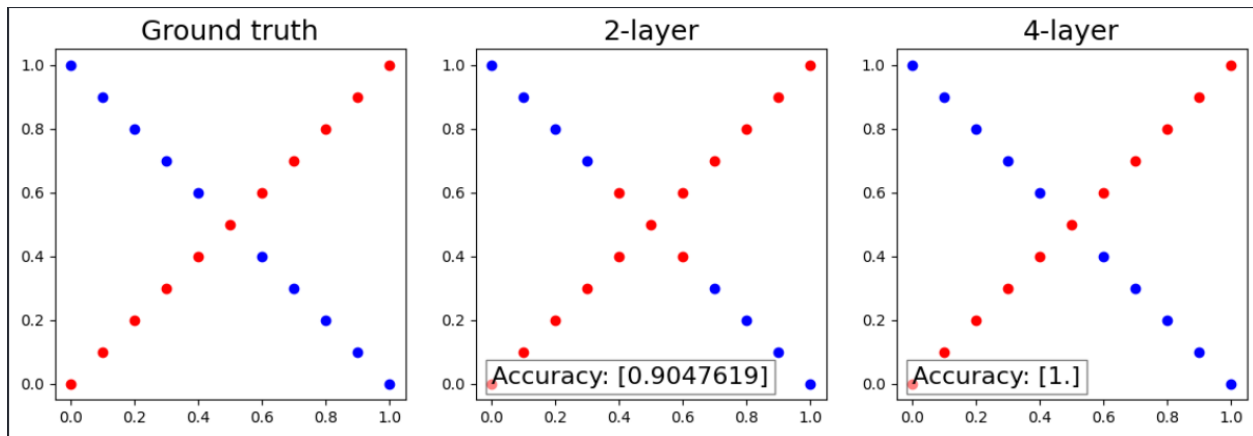


可以發現2 hidden layer表現比較好，但若調高epoch 還是可以都達到100%，只是在較低epoch 2 hidden layer performance比較好，有可能是 4 hidden layer的複雜度太高

- XOR data :

```
x_xor,y_xor =generate_XOR_easy()  
epochs = 8500  
model1 = Model(learning_rate=1e-2)  
model2 = Model_4layer(learning_rate=1e-2)  
  
compare(x_xor,y_xor,model1,model2,"2-layer","4-layer",epochs)
```





可以發現2 hidden layer表現比較好，但若調高epoch 還是可以都達到100%，只是在較低epoch 2 hidden layer performance比較好，有可能是 4 hidden layer的複雜度太高

5. Extra

A. Implement different optimizers.(Momentum)

Starting at θ^0

Movement $m^0 = 0$

Compute gradient g^0

Movement $m^1 = \lambda m^0 - \eta g^0$

Move to $\theta^1 = \theta^0 + m^1$

Compute gradient g^1

Movement $m^2 = \lambda m^1 - \eta g^1$

```

if(self.optimizer == "SGD"):
    #update
    self.W3 -= self.learning_rate * np.dot(self.hidden2.T,grad)
    self.b3 -= self.learning_rate * np.sum(grad,axis=0)
elif(self.optimizer == "Momentum"):
    self.v_W3 = self.decay *self.v_W3 + self.learning_rate * np.dot(self.hidden2.T,grad)
    self.v_b3 =self.decay *self.v_b3 + self.learning_rate * np.sum(grad,axis=0)
    self.W3 -= self.v_W3
    self.b3 -= self.v_b3

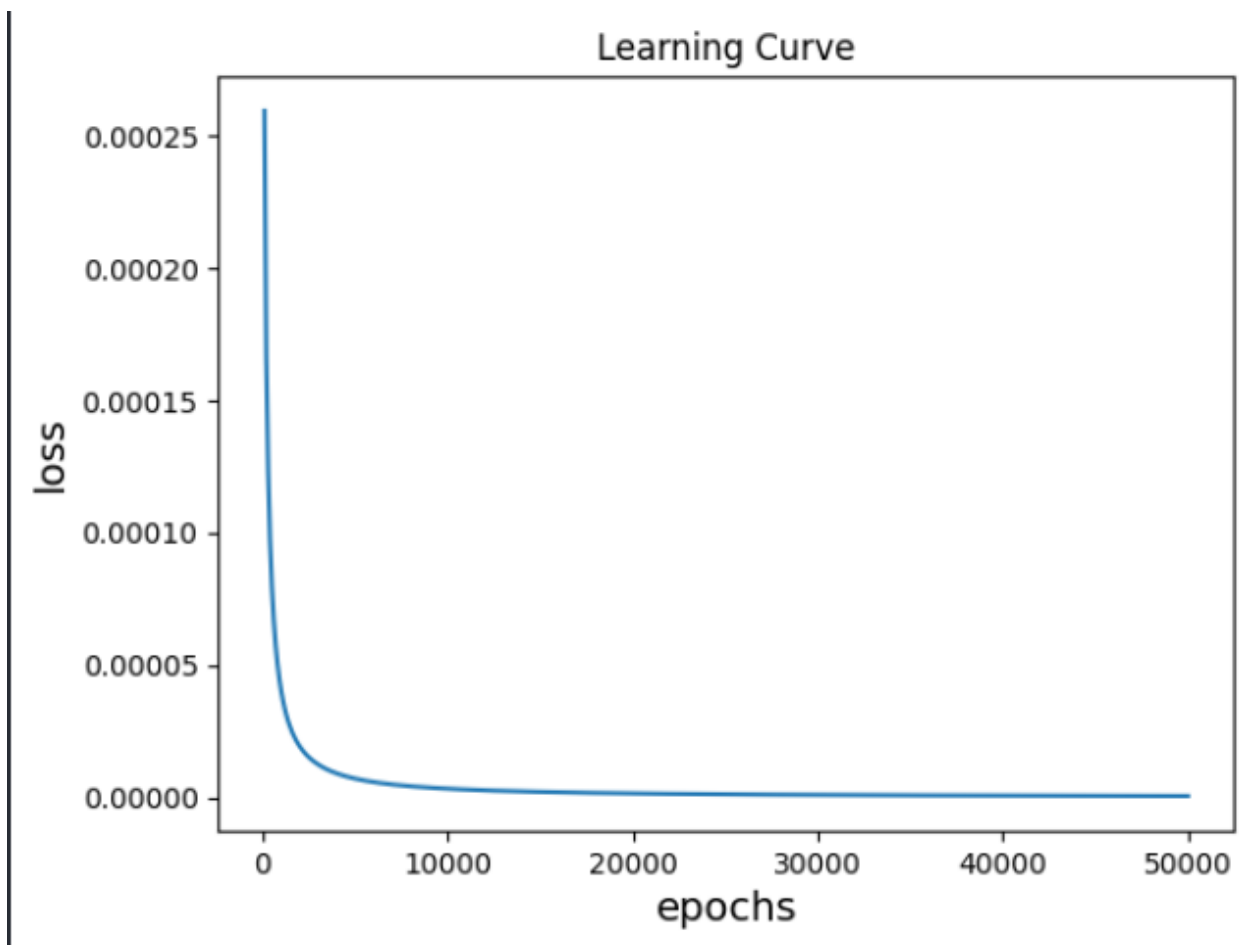
```

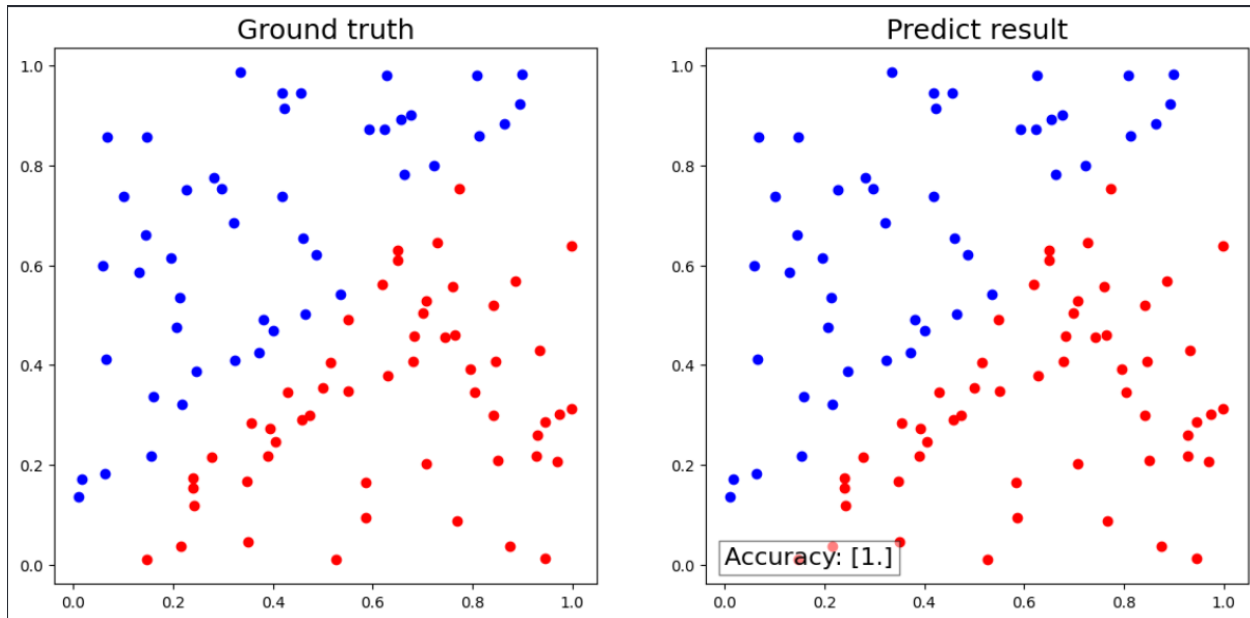
- Linear Data

```

x,y =generate_linear()
epochs = 50000
model =Model(optimizer="Momentum")
show(x,y,model,epochs)

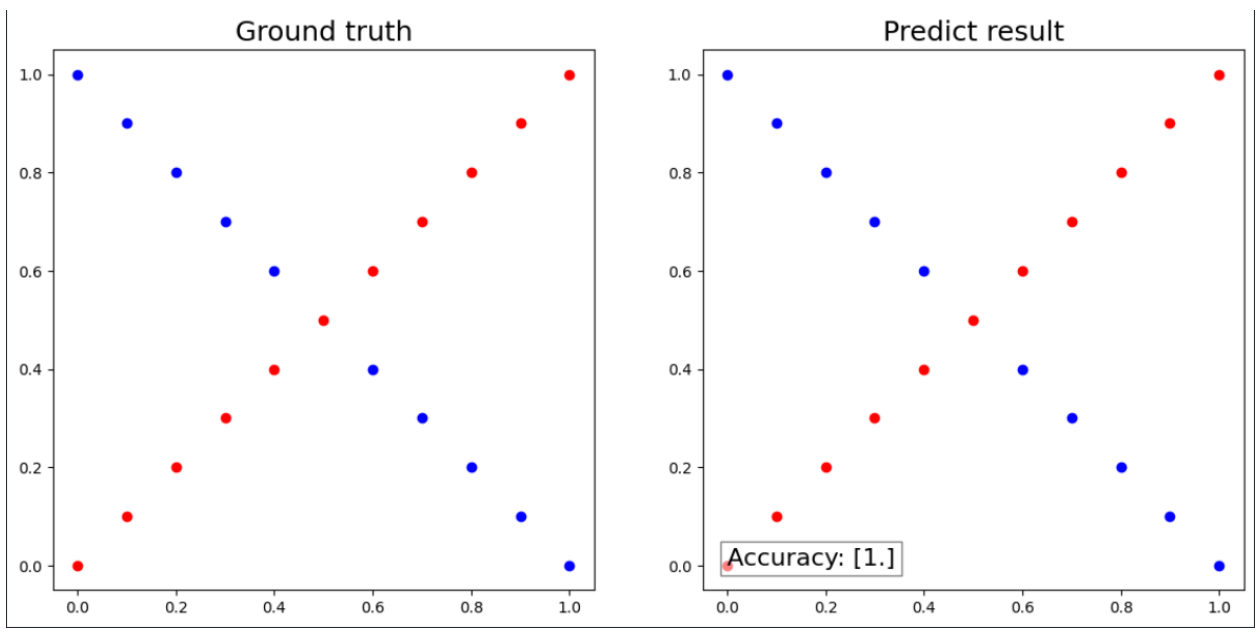
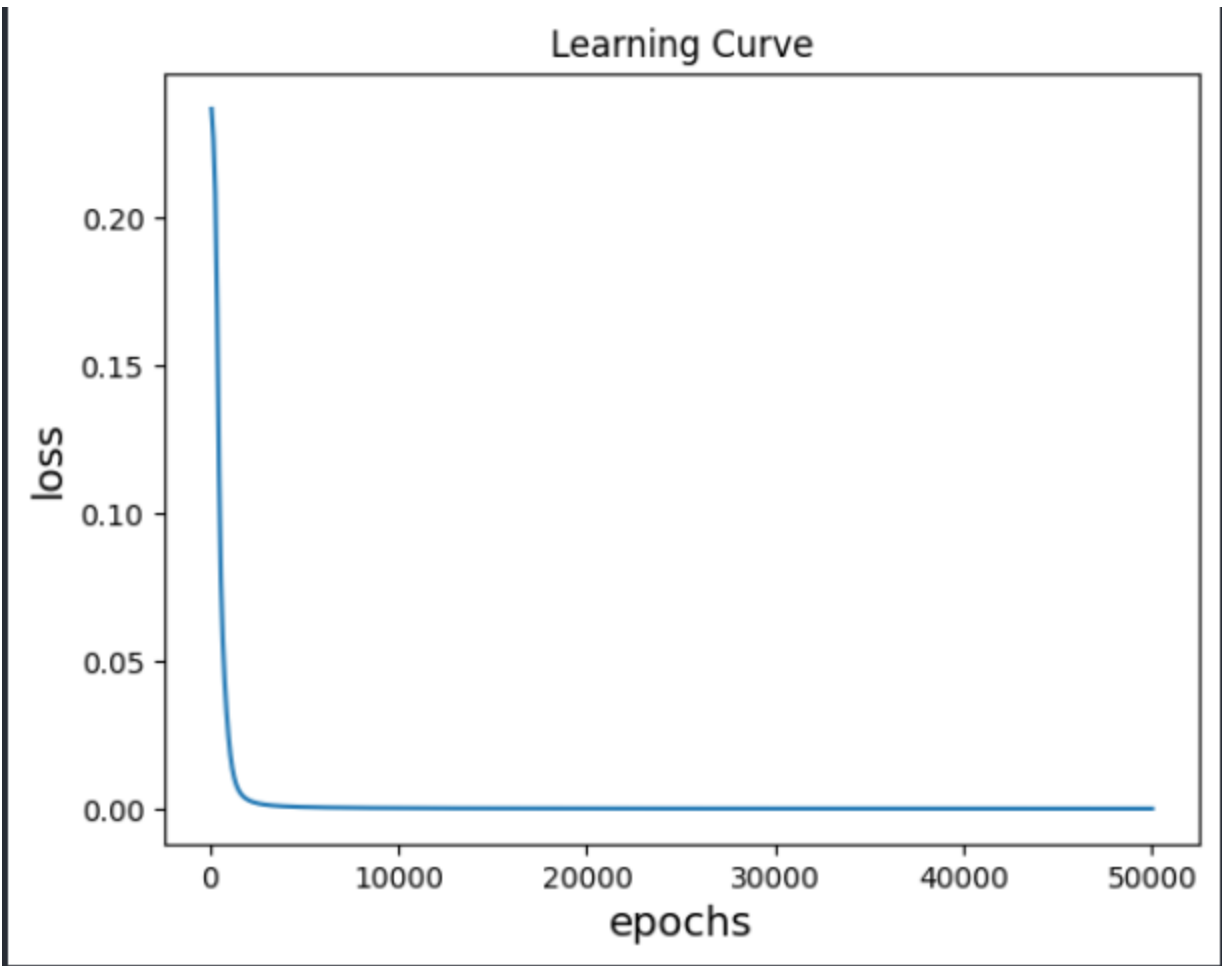
```





- XOR Data

```
x_xor,y_xor =generate_XOR_easy()  
epochs = 50000  
model= Model(optimizer="Momentum" ,learning_rate=1e-2)  
show(x_xor,y_xor,model,epochs)  
✓ 16.5s
```





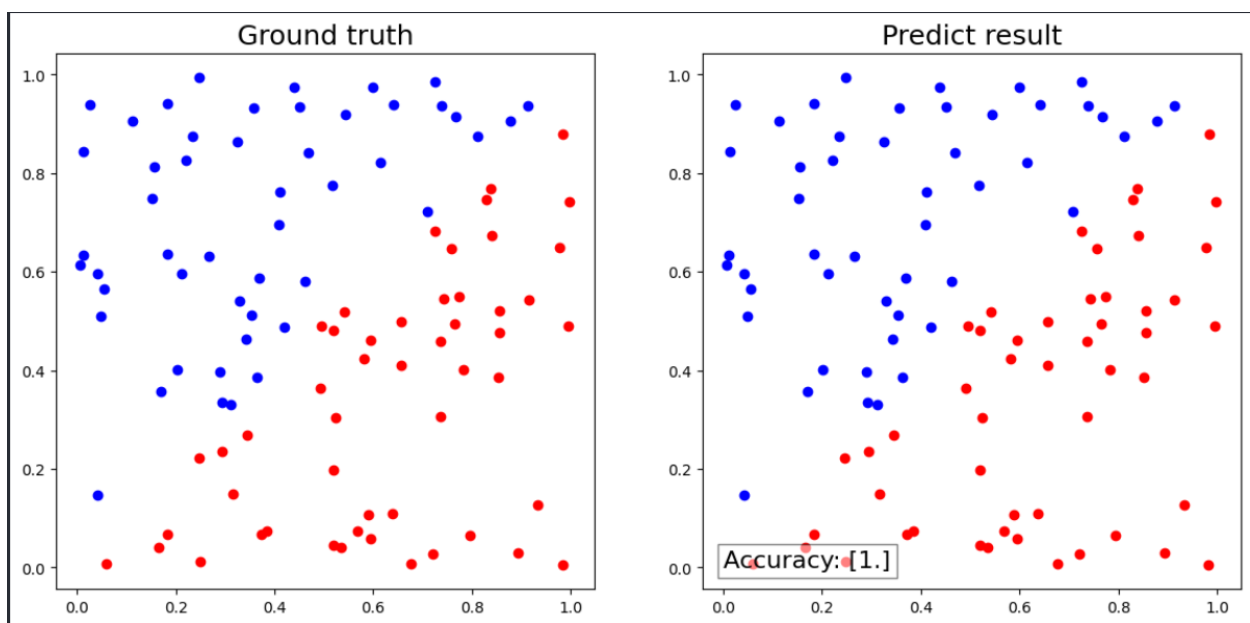
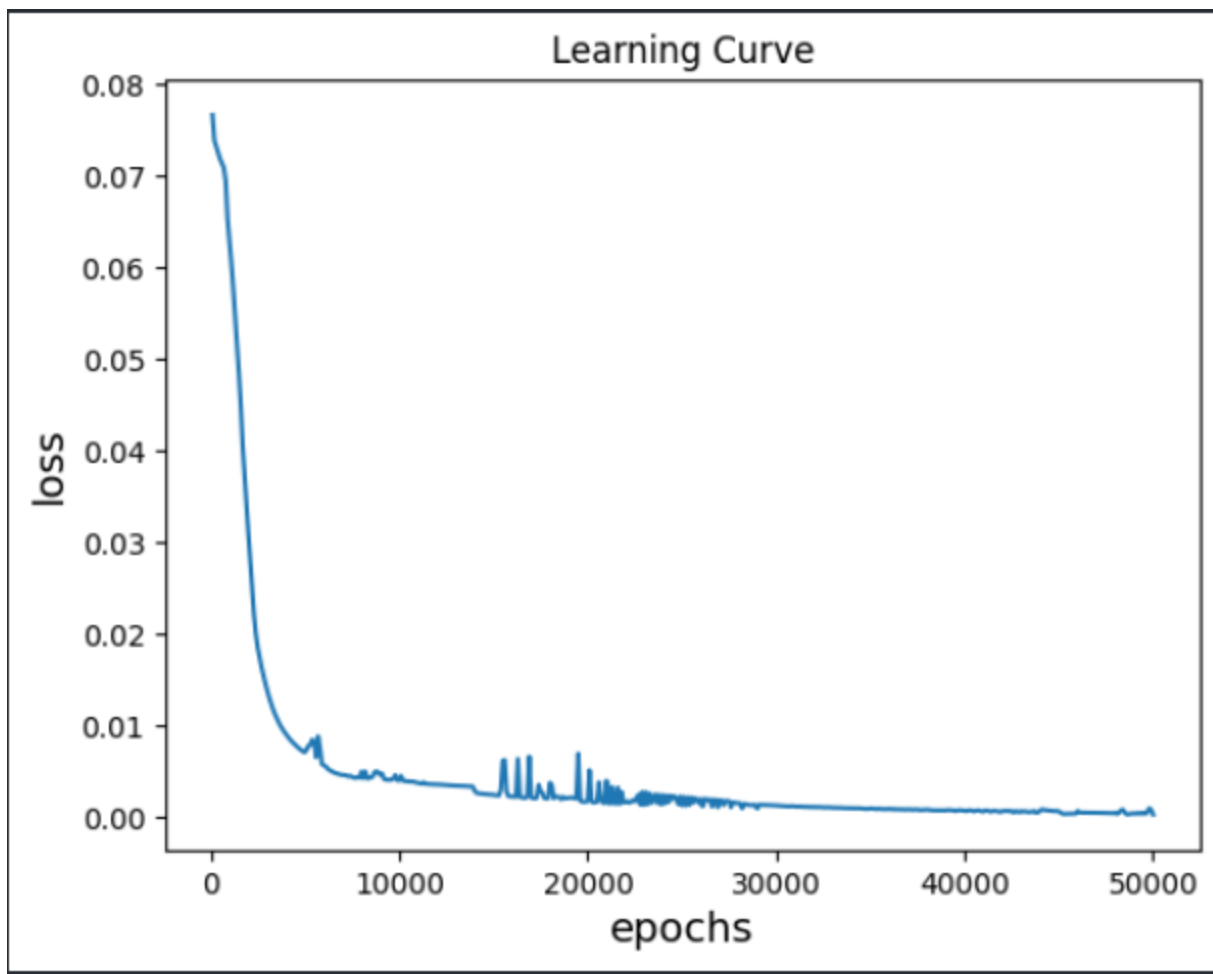
使用momentum，兩種Data Set皆可以達到100%

B. Implement different activation functions. (Relu)

```
def ReLu(x):  
    return np.where(x>0,x,0)  
def Derivative_ReLu(x):  
    return np.where(x>0,1,0)
```

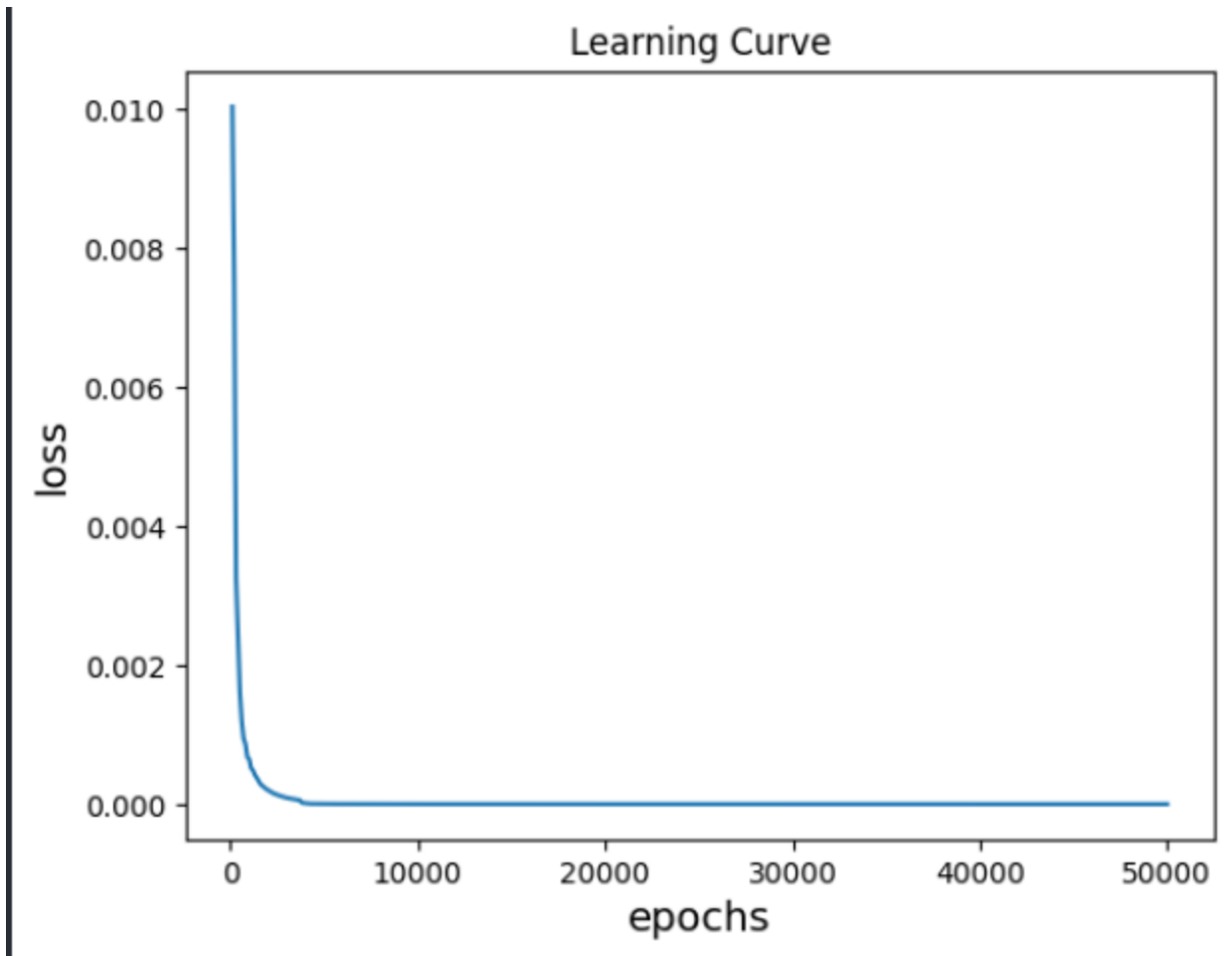
- Linear Data

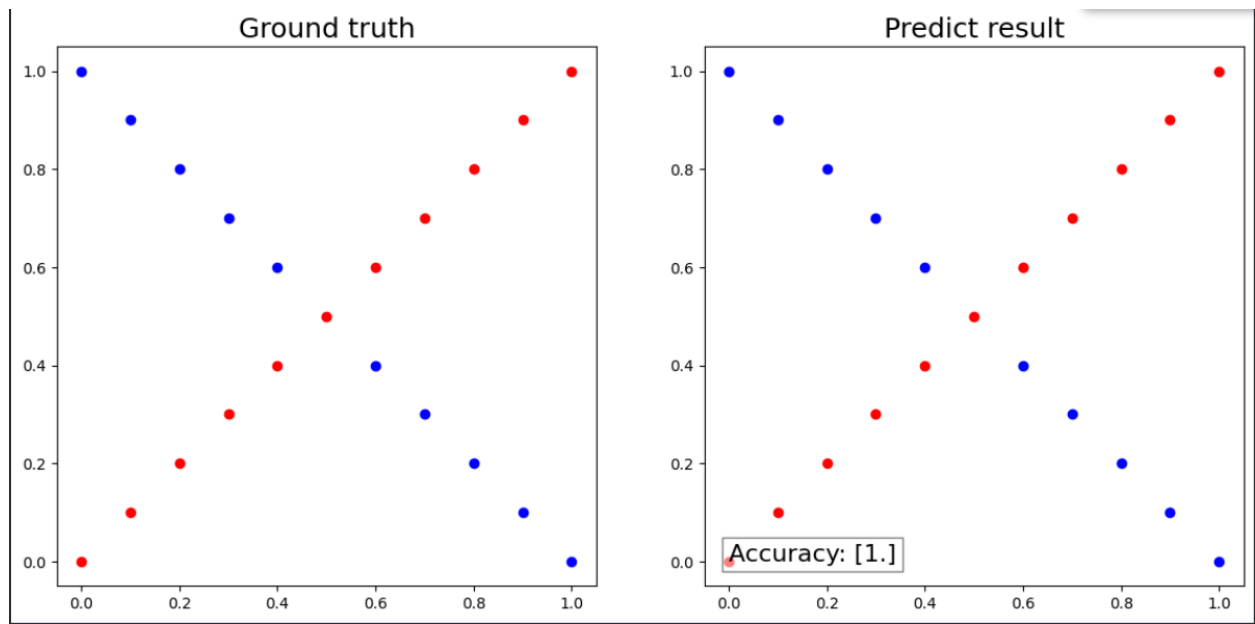
```
x,y =generate_linear()  
epochs = 50000  
model = Model(activate= "ReLu" ,learning_rate=1e-3)  
# training  
show(x,y,model,epochs)
```



- XOR Data

```
x_xor,y_xor =generate_XOR_easy()  
epochs = 50000  
model = Model(activate= "ReLu" ,learning_rate=1e-2)  
# training  
show(x_xor,y_xor,model,epochs)
```





ReLU 也能達到100%accracy